

Introdução

Autor: Clodoaldo A M Lima

9 de novembro de 2025

Sumário

- 1 Conteúdo Programático
- 2 Revisão de Álgebra Linear
- 3 Probabilidade
- 4 Revisão de Probabilidade
- 5 Revisão de Otimização Não Linear
- 6 Conceitos básicos
- 7 Modelos Lineares
- 8 Modelos Lineares Kernelizados
- 9 Modelos Lineares Regularizados
- 10 Modelos Lineares Regularizados
- 11 Introdução e Motivação
- 12 Formulação Matemática
- 13 Métodos Numéricos
- 14 Pseudocódigo
- 15 Implementação em Python
- 16 Conclusões e Extensões
- 17 Regressão Logística Binomial
- 18 Regressão Logística Multiclasse
- 19 Conclusões

Conteúdo Programático

● Aula 0 - Introdução

- Conteúdo Programático
- Revisão de Álgebra Linear e Otimização não Linear
- Conceitos básicos

● Aula 1 - Modelos Lineares

- Modelos Lineares, Modelos Lineares Regularizados, Modelos Lineares Kernelizados
- Regressão Logística Binomial e Multinomial
- Exercícios Computacionais

● Aula 2 - Redes Neurais Artificiais FeedForward

- Neurônio Biológico
- Rede Neural Perceptron Multicamadas
- Algoritmo *Backpropagation*
- Exercícios Computacionais

● Aula 3 - Redes Neurais Artificiais Recorrentes

- Rede de Hopfield
- Mapeamento de entrada e saída
 - Rede Neural com Recorrência Local
 - Rede Neural com Recorrência Global
 - Rede Neural com Recorrência Externa
 - Rede Neural Totalmente Recorrente

● Aula 4 - Máquinas de Vetores Suporte

- Abordagem Tradicional
 - Problema de Classificação: Formulação Primal e Dual
 - Problema de Regressão: Formulação Primal e Dual
- Least Square Support Vector Machine
- Relevance Vector Machine
- Outras variações
- Exercício computacional

● Aula 5 - Ensemble e Mistura de Especialistas

- Ensemble
 - Geração, Seleção e Combinação
- Mistura de Especialista
 - Formulação
 - Algoritmo EM
- Exercícios Computacionais

● Aula 6 - Redes Neurais Convolucionais

- Conceitos básicos: convolução, pooling, strider
- Arquiteturas
- Arquiteturas Recentes
- Exercícios Computacionais

● Aula 7 - Autoencoders

- Conceitos básicos
- Autoencoder Variacional
- Exercícios Computacionais

● Aula 8 - Rede Neural LSTM (Long Short-Term Memory)

- Conceitos básicos
- Variações
- Exercícios Computacionais

● Aula 9 - Redes Adversárias Generativas

- Conceitos básicos
- Variações
- Exercícios Computacionais

- Aula 10 - Técnicas de Clusterização - Parte 1
 - Conceitos básicos
 - Algoritmo de Particionamento
 - Algoritmo baseado em modelo
- Aula 11 - Técnicas de Clusterização - Parte 2
 - Conceitos básicos
 - Algoritmo baseado em Grid
- Aula 12 - Aprendizado por Reforço - Parte 1
 - Conceitos básicos: Agente, Ambiente
 - Bandits
 - Processo de Decisão de Markov e Programação Dinâmica
 - Monte Carlos
- Aula 13 - Aprendizado por Reforço - Parte 2
 - Aprendizado TD
 - Modelos e Planejamento
 - Métodos de Aproximação
 - Aproximação de Política
- Aula 13 - Aprendizado por Reforço - Parte 3
 - Aprendizado TD
 - Modelos e Planejamento
 - Métodos de Aproximação
 - Aproximação de Política

Revisão de Álgebra Linear

Um vetor $\mathbf{x} \in \mathbb{R}^n$ representa um ponto ou direção em um espaço vetorial.

Operações básicas:

- Soma: $\mathbf{x} + \mathbf{y}$
- Multiplicação escalar: $c\mathbf{x}$

- **Produto interno:**

$$\langle x, y \rangle = x^T y = \sum_{i=1}^n x_i y_i$$

$$\langle x, y \rangle = x^T y = \|x\| \|y\| \cos \theta$$

- **Norma Euclidiana:**

$$\|x\|_2 = \sqrt{x^T x} = \sqrt{\sum_{i=1}^n x_i^2}$$

- **Vetor unitário:**

$$\hat{x} = \frac{x}{\|x\|_2}, \quad \|\hat{x}\|_2 = 1$$

- Produto de Matrizes: $(AB)_{ij} = \sum_k A_{ik} B_{kj}$

- Matriz Transposta: $(A^T)_{ij} = A_{ji}$

Uma matriz $\mathbf{A} \in \mathbb{R}^{m \times n}$ representa uma transformação linear:

$$\mathbf{y} = \mathbf{A}\mathbf{x}$$

Operações importantes:

- Transposição: $\mathbf{A}^T$
- Produto: $\mathbf{AB}$
- Inversa: $\mathbf{A}^{-1}$ (quando $\mathbf{A}$ é quadrada e não singular)

Pseudo-Inversa de Matrizes

Quando $\mathbf{A}$ não é invertível, usamos a **pseudo-inversa**.

Pseudo-inversa à direita

$$\mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$$

Válida se $\mathbf{A}$ tem posto completo por colunas.

Pseudo-inversa à esquerda

$$\mathbf{A}^+ = \mathbf{A}^\top (\mathbf{A} \mathbf{A}^\top)^{-1}$$

Válida se $\mathbf{A}$ tem posto completo por linhas.

Pseudo-Inversa de Moore–Penrose

Definida via SVD:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^{\top} \Rightarrow \mathbf{A}^{+} = \mathbf{V}\mathbf{\Sigma}^{+}\mathbf{U}^{\top}$$

Propriedades:

- $\mathbf{A}\mathbf{A}^{+}\mathbf{A} = \mathbf{A}$
- $\mathbf{A}^{+}\mathbf{A}\mathbf{A}^{+} = \mathbf{A}^{+}$
- $(\mathbf{A}\mathbf{A}^{+})^{\top} = \mathbf{A}\mathbf{A}^{+}$

Propriedade: Transposta da Inversa

Proposição: Se A é uma matriz quadrada e invertível, então:

$$(A^{-1})^T = (A^T)^{-1}$$

Demonstração:

$$A^{-1}A = I$$

Tomando a transposta de ambos os lados:

$$(A^{-1}A)^T = I^T$$

Usando a propriedade $(AB)^T = B^T A^T$:

$$A^T(A^{-1})^T = I$$

Logo, $(A^{-1})^T$ é a inversa de A^T , isto é:

$$(A^{-1})^T = (A^T)^{-1}$$

Exemplo Numérico:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad A^{-1} = \begin{bmatrix} -2 & 1 \\ 1.5 & -0.5 \end{bmatrix}$$

- A **probabilidade** é uma medida associada a eventos de um espaço amostral Ω .

$$P: \mathcal{F} \rightarrow [0, 1], \quad P(\Omega) = 1.$$

- Para eventos $A, B \subseteq \Omega$:

$$0 \leq P(A) \leq 1, \quad P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

- Se $A_1, A_2, \dots, A_k$ são mutuamente exclusivos:

$$P\left(\bigcup_{i=1}^k A_i\right) = \sum_{i=1}^k P(A_i).$$

Probabilidade Conjunta e Condicional

- A **probabilidade conjunta** de dois eventos A e B :

$$P(A, B) = P(A \cap B).$$

- A **probabilidade condicional** de A dado B :

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{para } P(B) > 0.$$

- Pela definição:

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A).$$

- Dois eventos são **independentes** se:

$$P(A \cap B) = P(A)P(B).$$

Esperança (Valor Esperado)

Definição: Seja X uma variável aleatória com distribuição P .

$$\mathbb{E}[X] = \begin{cases} \sum x P(X = x), & (\text{discreta}) \\ \int_{-\infty}^{\infty} x f_X(x) dx, & (\text{contínua}) \end{cases}$$

Propriedades básicas:

- Linearidade: $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$
- Se c é constante: $\mathbb{E}[c] = c$
- Esperança condicional:

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X|Y]].$$

Propriedades da Esperança e Exemplo

- Para funções $g(X)$:

$$\mathbb{E}[g(X)] = \begin{cases} \sum_x g(x) P(X = x), & \text{(discreta)} \\ \int g(x) f_X(x) dx, & \text{(contínua)} \end{cases}$$

- Se X, Y são independentes:

$$\mathbb{E}[XY] = \mathbb{E}[X] \mathbb{E}[Y].$$

- **Exemplo:** Seja $X \sim U(0, 1)$. Então:

$$\mathbb{E}[X] = \int_0^1 x dx = \frac{1}{2}, \quad \mathbb{E}[X^2] = \int_0^1 x^2 dx = \frac{1}{3}.$$

Variância e Desvio Padrão

Definição:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Desvio padrão: $\sigma_X = \sqrt{\text{Var}(X)}$

Propriedades:

- $\text{Var}(aX + b) = a^2 \text{Var}(X)$
- $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)$
- Se X, Y independentes: $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$

Exemplo: Se $X \sim U(0, 1)$:

$$\text{Var}(X) = \frac{1}{3} - \left(\frac{1}{2}\right)^2 = \frac{1}{12}.$$

Definições:

$$\text{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}, \quad -1 \leq \rho_{XY} \leq 1.$$

Propriedades:

- Se X, Y independentes, então $\text{Cov}(X, Y) = 0$.
- Covariância linear:

$$\text{Cov}(aX + bY, cZ + dW) = ac\text{Cov}(X, Z) + ad\text{Cov}(X, W) + bc\text{Cov}(Y, Z) + bd\text{Cov}(Y, W).$$

Distribuição Conjunta e Marginal

Distribuição conjunta: Para variáveis X, Y :

$$P_{X,Y}(x, y) = P(X = x, Y = y) \quad \text{ou} \quad f_{X,Y}(x, y)$$

Distribuições marginais:

$$P_X(x) = \sum_y P_{X,Y}(x, y), \quad f_X(x) = \int f_{X,Y}(x, y) dy.$$

Distribuição condicional:

$$P_{X|Y}(x|y) = \frac{P_{X,Y}(x, y)}{P_Y(y)}, \quad f_{X|Y}(x|y) = \frac{f_{X,Y}(x, y)}{f_Y(y)}.$$

Lei da Esperança Total

Seja Y uma partição discreta do espaço amostral, então:

$$\mathbb{E}[X] = \sum_y \mathbb{E}[X|Y=y] P(Y=y).$$

Na forma contínua:

$$\mathbb{E}[X] = \int \mathbb{E}[X|Y=y] f_Y(y) dy.$$

Exemplo: Se $X|Y=y \sim \mathcal{N}(y, 1)$ e $Y \sim \mathcal{N}(0, 1)$, então

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X|Y]] = \mathbb{E}[Y] = 0.$$

Teorema de Bayes

Definição:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}, \quad \text{com } P(B) > 0.$$

onde

$$P(B) = \sum_i P(B|A_i)P(A_i)$$

para eventos $A_1, \dots, A_k$ que formam uma partição do espaço.

Versão contínua:

$$f_{X|Y}(x|y) = \frac{f_{Y|X}(y|x)f_X(x)}{f_Y(y)}.$$

Exemplo: Dado um teste médico com sensibilidade 0.99, especificidade 0.95 e prevalência $P(\text{Doença}) = 0.01$:

$$P(\text{Doença}|\text{Teste}^+) = \frac{0.99 \times 0.01}{0.99 \times 0.01 + 0.05 \times 0.99} \approx 0.167.$$

Revisão de Otimização Não Linear

Método do Gradiente

Atualiza parâmetros na direção oposta ao gradiente:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f(\mathbf{x}_k)$$

onde α_k é o passo (learning rate).

Método de Newton

Usa a Hessiana $\mathbf{H}$:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \mathbf{H}^{-1} \nabla f(\mathbf{x}_k)$$

Convergência rápida perto do ótimo.

Métodos Quase-Newton e Gradiente Conjugado

- Quase-Newton (BFGS): aproxima $\mathbf{H}^{-1}$.
- Gradiente Conjugado: evita calcular $\mathbf{H}$, útil em grandes dimensões.

Conceitos básicos

- O aprendizado de máquina envolve a extração de conhecimento diretamente de dados disponíveis, visando “modelar a porção do mundo” de interesse, realizar previsões e dar suporte à tomada de decisão, dentre outras aplicações.
- Ele está fundamentado em teoria de probabilidade, estatística e otimização, sustenta técnicas indispensáveis em soluções para big data, ciência de dados, reconhecimento de padrões e recuperação de informação, além de fomentar os principais progressos recentes em inteligência artificial.
- Em termos de aplicações práticas, técnicas de aprendizado de máquina se tornaram ingredientes críticos em reconhecimento de padrões, visão computacional, processamento de linguagem natural e robótica.

Introdução

Objetivo de aprendizado de Máquina

O objetivo do aprendizado de máquina é construir modelos computacionais que podem adaptar-se e aprender a partir da experiência (MITCHELL,1997).

Aprendizado Indutivo - Segundo MITCHELL (1997)

"Um programa de computador aprende a partir de um elenco de experiências E , relacionadas a uma classe de tarefas T e dispondo de uma medida de desempenho M , se seu desempenho medido por M junto a tarefa T melhora com o elenco de experiências E ."

Exemplo

No contexto de redes neurais artificiais, o seu processo de treinamento pode, então, ser caracterizado como aprendizado indutivo, sendo que o uso posterior da rede neural treinada para classificação, regressão ou agrupamento de dados e geralmente denominado de processo de inferência dedutiva

Tipos de Raciocínio

Raciocínio Indutivo

- Conhece: $p(a,b)$, $p(a,d)$, $p(d,e)$, $p(d,g)$, $p(e,f)$
- Observa: $a(a,e)$ e $a(d,f)$
- Aprende: $p(X,Y) \cap a(Y,Z) \rightarrow p(X,Z)$

Raciocínio Abdutivo

- Conhece: $p(a,b)$, $p(a,d)$, $p(d,e)$, $p(d,g)$, $p(e,f)$, $p(X,Y) \cap p(Y,Z) \rightarrow a(X,Z)$
- Observa: $a(a,c)$
- Explica: $p(b,c)$ ou $p(d,c)$

Raciocínio Dedutivo

- Conhece: $p(a,b)$, $p(a,d)$, $p(b,c)$, $p(d,e)$, $p(d,g)$, $p(e,f)$, $p(X,Y) \cap p(Y,Z) \rightarrow a(X,Z)$
- Conclui: $a(a,c)$, $a(a,e)$, $a(a,g)$ e $a(d,f)$

Principais módulos do aprendizado de máquina

- Em termos formais, técnicas de aprendizado de máquina são compostas por:
- Um conjunto de dados: pode ser particionado nos conjuntos de treinamento, validação e teste, que serão melhor descritos mais adiante.
- Um modelo matemático (modelo de aprendizado) para a síntese de um mapeamento multidimensional de entrada-saída, que possui algum grau de plasticidade ou flexibilidade a partir da presença de parâmetros livres.
- Uma função de perda ou de custo que mede o desempenho do modelo de aprendizado: por exemplo, pode ser o erro quadrático médio em problemas de regressão e a taxa de erro de classificação em problemas de classificação.
- Um procedimento de otimização de parâmetros livres do modelo visando minimizar as perdas ou o custo ao se caminhar pela superfície de erro, que pode ou não ser convexa.

Aprendizado de Máquina

Em termos práticos, algoritmos de aprendizado de máquina têm como objetivo descobrir o relacionamento entre as variáveis de um sistema (entrada/saída) a partir de dados amostrados (CHERKASSKY & MULIER, 2007).

Sendo assim, eles não são necessários quando os relacionamentos entre todas as variáveis do problema (entrada/saída) são completamente compreendidos. Este definitivamente não é o caso de muitos dos problemas reais com os quais nos defrontamos em nosso dia-a-dia.

Há três frentes principais em aprendizado de máquina

- Aprendizado supervisionado
- Aprendizado de não supervisionado
- Aprendizado por reforço

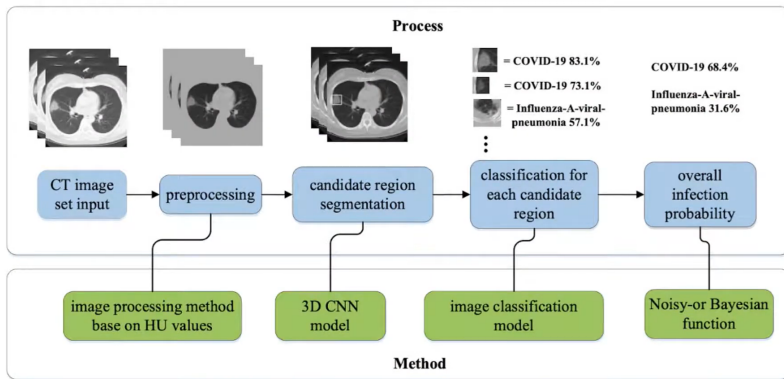
Principais vertentes do aprendizado de máquina

- Em aprendizado supervisionado, há uma quantidade finita e possivelmente ruidosa de dados rotulados, indicando (com limitações) o comportamento desejado para a máquina de aprendizado. Quando as saídas desejadas são discretas ou categóricas, geralmente se fala de problemas de classificação supervisionada. Já para o caso de saídas contínuas ou numéricas, tem-se problemas de regressão. De qualquer modo, ambos envolvem a aproximação de mapeamentos multidimensionais contínuos de entrada-saída, com múltiplos estímulos de entrada e múltiplas saídas desejadas (MIMO: Multiple Input, Multiple Output).
- Técnicas de aprendizado não-supervisionado buscam extrair conhecimento a partir de dados não-rotulados, por exemplo, realizando clusterização e estimação de densidade (lei de distribuição que responde pela geração dos dados). A clusterização visa agrupar, ou seja, classificar numa mesma classe, objetos que apresentem atributos similares (requer o emprego de métricas de distância),

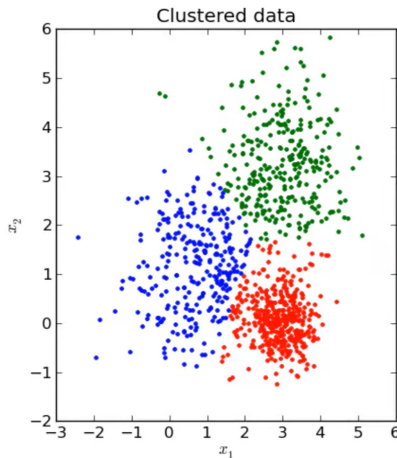
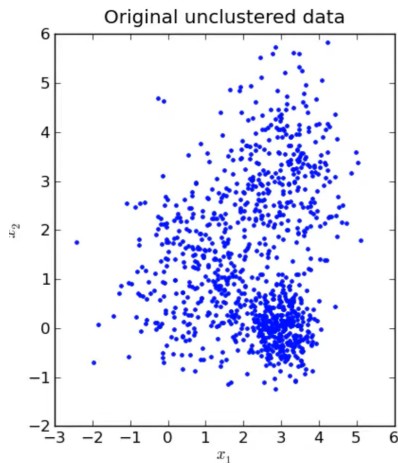
Principais vertentes do aprendizado de máquina

- Em aprendizado por reforço, não há um claro indicativo do que é certo e do que é errado o tempo todo, como ocorre em aprendizado supervisionado, mas esporadicamente é recebido um grau de avaliação de desempenho (recompensa ou punição) de uma tarefa caracterizada por uma sequência de decisões. O objetivo então é maximizar a recompensa acumulada a longo prazo, ou seja, encontrar a melhor estratégia de tomada de decisões sequenciais, também chamada de política ótima. Trata-se de uma técnica que geralmente recorre a ambientes simulados em computador. É impressionante o sucesso do aprendizado por reforço, por exemplo, em jogos de tabuleiro, particularmente pelo fato da máquina poder jogar contra si própria enquanto aprende. Lembre-se que o tempo de uma simulação computacional em um ambiente virtual pode ser muito acelerado em relação ao tempo físico de ocorrência do experimento no mundo real.

Aprendizado Supervisionado



Aprendizado Não Supervisionado



Aprendizado Por Reforço



Crescimento de Aprendizado de Máquina

Aprendizado de Máquina e uma abordagem muito utilizada para

- Reconhecimento de fala, processamento de linguagem natural
- Visão computacional
- Análises médica
- Biologia computacional
- Redes de sensores

Esta tendência é acelerada por:

- Big Data
- Melhoria de Algoritmos de aprendizado de máquina
- Computadores mais rápido

Definição

- Dado um conjunto de treinamento $f(x_i; y_i) \quad i = 1; \dots; N$
- Encontrar uma boa aproximação para $f: X \rightarrow Y$

Exemplos: O que representam X e Y

- Detecção Spam
 - Mapear texto para (Spam, Não-Spam)
- Reconhecimento de Dígito
 - Mapear pixels para 0,1,2,3,4,5,6,7,8,9
- Predição de Ações
 - Mapear preços históricos para $\mathbb{R}$ (número real)

Problema de Aprendizado de Máquina

Conjunto de Dados

Exemplo	x_1	x_2	x_3	x_4	y
1	0	0	1	0	0
2	0	1	0	0	0
3	0	0	1	1	1
4	1	0	0	1	1
5	0	1	1	0	0
6	1	1	0	0	0
7	0	1	0	1	0

Nosso objetivo é encontrar uma função

$f : XY$

$$X = \{0, 1\}^4$$

$$Y = \{0, 1\}$$

Questão 1

Como definir o espaço de hipótese, o conjunto possíveis de f

Questão 2

Como encontrar melhor f no espaço de hipótese

Espaço de Hipoteses mais gerais

Considere todas as possíveis funções booleanas sobre 4 características como entrada

2^{16} hipóteses possíveis
 2^9 são consistente com
nosso conjunto de dados.
Como escolher a melhor
hipótese?

x_1	x_2	x_3	x_4	y
0	0	0	0	?
0	0	0	1	?
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	?
1	0	0	0	?
1	0	0	1	1
1	0	1	0	?
1	0	1	1	?
1	1	0	0	0
1	1	0	1	?
1	1	1	0	?
1	1	1	1	?

Conjunto de Dados

exemplo	x_1	x_2	x_3	x_4	y
1	0	0	1	0	0
2	0	1	0	0	0
3	0	0	1	1	1
4	1	0	0	1	1
5	0	1	1	0	0
6	1	1	0	0	0
7	0	1	0	1	0

Espaço de Hipoteses mais gerais

Considere todas as funções booleanas conjuntivas

16 hipóteses possíveis
nenhuma é consistente
com nosso conjunto de
dados
Como escolher a melhor
hipótese?

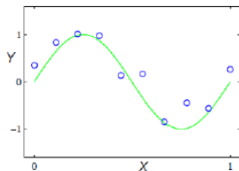
Rule	Counterexample
$\Rightarrow y$	1
$x_1 \Rightarrow y$	3
$x_2 \Rightarrow y$	2
$x_3 \Rightarrow y$	1
$x_4 \Rightarrow y$	7
$x_1 \wedge x_2 \Rightarrow y$	3
$x_1 \wedge x_3 \Rightarrow y$	3
$x_1 \wedge x_4 \Rightarrow y$	3
$x_2 \wedge x_3 \Rightarrow y$	3
$x_2 \wedge x_4 \Rightarrow y$	3
$x_3 \wedge x_4 \Rightarrow y$	4
$x_1 \wedge x_2 \wedge x_3 \Rightarrow y$	3
$x_1 \wedge x_2 \wedge x_4 \Rightarrow y$	3
$x_1 \wedge x_3 \wedge x_4 \Rightarrow y$	3
$x_2 \wedge x_3 \wedge x_4 \Rightarrow y$	3
$x_1 \wedge x_2 \wedge x_3 \wedge x_4 \Rightarrow y$	3

Conjunto de Dados

exemplo	x_1	x_2	x_3	x_4	y
1	0	0	1	0	0
2	0	1	0	0	0
3	0	0	1	1	1
4	1	0	0	1	1
5	0	1	1	0	0
6	1	1	0	0	0
7	0	1	0	1	0

Regressão

Conjunto de dados: 10 pontos (X, Y) gerados a partir da função seno com ruído



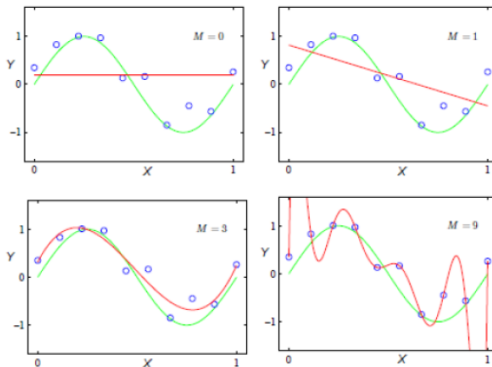
Regressão

$$f : X \rightarrow Y$$

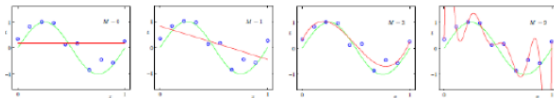
$$X = \mathcal{R}$$

$$Y = \mathcal{R}$$

Qual é o melhor grau do polinômio M ?



Regressão

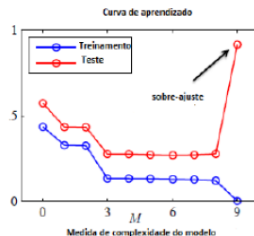


Nós medimos o erro usando uma função perda $L(y, \hat{y})$
Para regressão, uma escolha comum é perda quadrada:

$$L(y_i, f(x_i)) = (y_i - f(x_i))^2$$

A perda empírica da função f aplicada para dados de treinamento é então

$$\frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i))^2$$



Princípio de Occam's Razor

William of Occam: Monge viveu no século 14

Princípio da parcimônia:

"One should not increase, beyond what is necessary, the number entities required to explain anything"

- Quando várias soluções estão disponíveis para uma dado problema, nós devemos selecionar aquela mais simples
- Mas o que nós queremos dizer por simples?
- Nós usaremos o conhecimento a priori do problema para solucionar e definir o que é uma solução simples
- Exemplo de um conhecimento a priori: suavidade

Modelos Lineares

- Modelos lineares são uma das abordagens estatísticas mais antigas e fundamentais.
- Aplicados em ciência, economia, engenharia e ciência de dados.
- Permitem prever variáveis dependentes com base em variáveis independentes.

Histórico:

- Carl Friedrich Gauss (1821): primeiros desenvolvimentos da regressão linear.
- Francis Galton: popularizou correlação e regressão em contextos biológicos.

Equação Geral do Modelo Linear

$$Y = Xw^T + \varepsilon \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Onde:

- Y = vetor de respostas ($N \times p$)
- X = matriz de preditores ($N \times d$)
- β = vetor de coeficientes ($p \times 1$)
- ε = erro aleatório ($n \times 1$), com $\mathbb{E}[\varepsilon] = 0$

$$w = (X^T X)^{-1} X^T y$$

Classificação

Em aprendizado supervisionado, é comum termos:

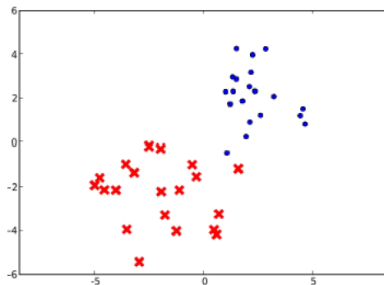
- Uma variável independente (ou variável-alvo, ou ainda resultado) Y
- Uma ou mais variáveis dependentes (ou covariantes) $X_1 \dots X_p$
- Um conjunto de observações, em que Y e X_i são observadas

Difere do não supervisionado em que neste último nem sempre se conhece Y

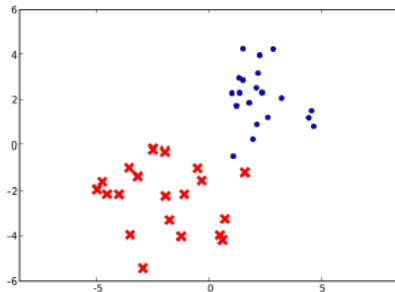
Na classificação, além do citado, temos um conjunto pré-definido de classes $C_1 \dots C_j$ às quais está associado ao valor y da variável Y

Queremos então um procedimento que possa ser aplicado a uma seqüência de casos, de modo a que novas instâncias possam ser associados a uma de um conjunto de classes pré-definidas ($Y = y \in \{C_1 \dots C_j\}$), com base em seu atributos observados $X_i = x_i$.

- Imagine que queremos separar os Xs e Os (exemplos pré-classificados nessas duas categorias)
 - Cada ponto é descrito por um número de atributos
 - No exemplo, 2: eixo x e y
 - Queremos achar um meio de classificar novos exemplos nessas categorias

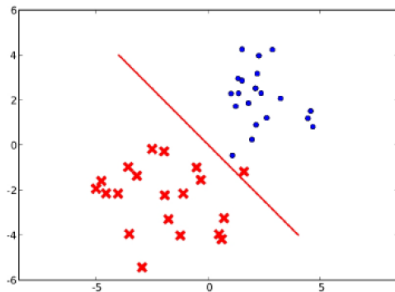


- Imagine que queremos separar os Xs e Os (exemplos pré-classificados nessas duas categorias)
 - Cada ponto é descrito por um número de atributos
 - No exemplo, 2: eixo x e y
 - Queremos achar um meio de classificar novos exemplos nessas categorias
- Como fazer?



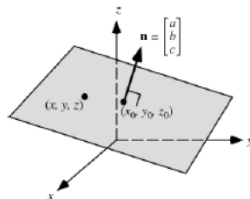
- Modo mais simples: divide os pontos com uma linha
– **classificação linear**

- A classificação é feita com base no valor de uma combinação linear dos atributos
- Em mais de 2 dimensões, são hiperplanos, não linhas



- O hiperplano irá obedecer à seguinte equação:
 $\omega_1 x_1 + \omega_2 x_2 + \dots \omega_d x_d + \omega_0 = 0$
 - Inclinação: determinada pelo vetor $(\omega_1, \dots, \omega_d)$ (sua normal)
 - Localização: determinada pelo **viés** ω_0
 - Geometricamente, o vetor de parâmetros é perpendicular ao hiperplano

$$\begin{aligned}\text{plano: } \vec{\omega} \cdot (\vec{x} - \vec{x}_0) &= 0 \\ \Rightarrow |\vec{\omega}| |\vec{x} - \vec{x}_0| \cos(\theta) &= 0 \\ \Rightarrow \theta &= 90^\circ \\ \text{e } \omega_0 &\equiv -\omega_1 x_{01} - \dots - \omega_d x_{0d}\end{aligned}$$

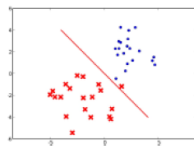


- Como classificar?

- Seja $f(\vec{x}) = \omega_1 x_1 + \omega_2 x_2 + \dots \omega_d x_d + \omega_0$

- Então

$$h(\vec{x}) = \text{sign}(f(\vec{x})) = \begin{cases} c & \text{se } f(\vec{x}) \geq 0 \\ \bar{c} & \text{se } f(\vec{x}) < 0 \end{cases}$$

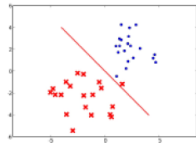


- Como classificar?

- Seja $f(\vec{x}) = \omega_1 x_1 + \omega_2 x_2 + \dots + \omega_d x_d + \omega_0$

- Então

$$h(\vec{x}) = \text{sign}(f(\vec{x})) = \begin{cases} c & \text{se } f(\vec{x}) \geq 0 \\ \bar{c} & \text{se } f(\vec{x}) < 0 \end{cases}$$



- Assim, dado um ponto $p = (x, y)$, e o plano

$\omega_1 x + \omega_2 y + \omega_0 = 0$, temos que:

$$h(\vec{p}) = \text{sign}(\omega_1 x + \omega_2 y + \omega_0) = \begin{cases} c & \text{se } f(\vec{p}) \geq 0 \\ \bar{c} & \text{se } f(\vec{p}) < 0 \end{cases}$$

- Exemplo:

- Considerare $\vec{x} = (x_1, \dots, x_d)$ os atributos de um cliente

- Seu crédito será aprovado se $\sum_{i=1}^d \omega_i x_i \geq \text{limiar}$

- Será negado se $\sum_{i=1}^d \omega_i x_i < \text{limiar}$

- Note que $h(x)$ pode ser reescrita como:

$$h(\vec{x}) = \text{sinar} \left(\left(\sum_{i=1}^d \omega_i x_i \right) - \text{limiar} \right)$$

Classificação Linear

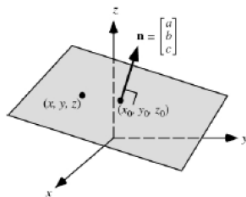
- Para fins práticos, podemos criar uma coordenada artificial $x_0 = 1$, e

$$\omega_1 x_1 + \omega_2 x_2 + \dots \omega_d x_d + \omega_0$$

$$\Rightarrow \left(\sum_{i=1}^d \omega_i x_i \right) + \omega_0$$

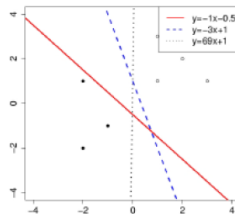
onde $\omega_0 = -\text{limiar}$. Com $x_0 = 1$ $h(\vec{x})$ pode ser então simplificada para

$$h(\vec{x}) = \text{sinar} \left(\sum_{i=0}^d \omega_i x_i \right)$$



Classificação Linear

- E qual o melhor hiperplano?
 - Definido pelo processo de aprendizado – exemplos são usados para se achar “bons” ω_i
 - Diferentes noções de “bom” existem $\rightarrow$ diferentes algoritmos
- Técnicas para classificação linear:
 - Redes Neurais (multi-layer perceptron)
 - Naïve Bayes
 - SVM
 - Regressão Logística



- Qualquer que seja o algoritmo, as hipóteses sempre obedecerão a

$$h(\vec{x}) = \text{sinal} \left(\left(\sum_{i=1}^d \omega_i x_i \right) - \text{limiar} \right)$$

- O que define a hipótese h é então a escolha de ω_i e limiar
- Isso é o que o algoritmo precisa variar para escolher a melhor hipótese

Regressão Linear

- Classificação:
 - Tenta classificar um exemplo em um conjunto de classes pré-determinadas
 - Ex:
 - Aprovação de crédito: (sim/não)
- Regressão:
 - Tenta descobrir o valor associado a um novo exemplo
 - A diferença está em que na regressão a variável-alvo é numérica e contínua
 - Ex:
 - Linha de crédito: quantos R\$ podem ser emprestados

- Considere o problema do crédito:

- Entrada: $x =$

idade	23
salário anual	R\$30.000,00
anos na residência	1
anos no trabalho	1
dívida atual	R\$15.000,00
...	...

- Hipótese:
$$h(\vec{x}) = \sum_{i=0}^d \omega_i x_i = \vec{\omega} \cdot \vec{x}$$

Regressão, pela saída real, e linear pela forma de tratamento da entrada ser linear (note a remoção da função *senal*)

- Dados de entrada:

$$(\vec{x}_1, y_1), (\vec{x}_2, y_2), \dots, (\vec{x}_N, y_N)$$

onde

$\vec{x}_i = \{x_j\}$ corresponde ao pedido de crédito feito pelo i -ésimo cliente (atributos)

$y_i \in \mathbb{R}$ ao montante concedido

- A regressão irá tentar replicar as decisões tomadas pelas pessoas
 - A questão é: com que precisão? Ou
 - Quão boa é a aproximação entre $h(\vec{x}) = \vec{\omega} \cdot \vec{x}$ e $f(\vec{x})$ (a função que hipoteticamente determina os dados reais)

- Medindo o erro:

- Importante para saber qual hipótese $h(\vec{x})$ melhor aproxima a função-alvo $f(\vec{x})$
- O algoritmo irá tentar minimizar o erro da amostra movendo-se de uma hipótese a outra
- No caso da regressão linear, usamos o erro quadrático $(h(\vec{x}) - f(\vec{x}))^2$
 - Se usarmos meramente a diferença entre previsto e real, valores positivos podem cancelar negativos
 - Calculamos então a média do erro da amostra para cada ponto (dos N) que temos:

$$E(h) = \frac{1}{N} \sum_{i=1}^N (h(\vec{x}_i) - y_i)^2$$

- Variação do método dos quadrados mínimos:
 - Com quadrados mínimos minimizamos a soma dos erros quadráticos
 - Aqui minimizamos a média
- Considere a expressão do erro:

$$E(h) = \frac{1}{N} \sum_{i=1}^N (h(\vec{x}_i) - y_i)^2$$

como $h(\vec{x}) = \vec{\omega} \cdot \vec{x}$, então temos

$$E(\vec{\omega}) = \frac{1}{N} \sum_{i=1}^N (\vec{\omega} \cdot \vec{x}_i - y_i)^2$$

Regressão Linear - Medindo o erro

- e...

$$\begin{aligned} E(\vec{\omega}) &= \frac{1}{N} \sum_{i=1}^N (\vec{\omega} \cdot \vec{x}_i - y_i)^2 \\ &= \frac{1}{N} \|X\vec{\omega} - \vec{y}\|^2 \end{aligned}$$

onde

$$X = \begin{bmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,d} \\ x_{2,0} & x_{2,1} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,0} & x_{N,1} & \cdots & x_{N,d} \end{bmatrix}$$

Atributos (0-d) de cada exemplo.

$$\vec{\omega} = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \vdots \\ \omega_d \end{bmatrix}$$

Pesos de cada atributo.

$$\vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}$$

Saída de cada exemplo.

- Por partes...

$$X\vec{\omega} = \begin{bmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,d} \\ x_{2,0} & x_{2,1} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,0} & x_{N,1} & \cdots & x_{N,d} \end{bmatrix} \begin{bmatrix} \omega_0 \\ \omega_1 \\ \vdots \\ \omega_d \end{bmatrix} = \begin{bmatrix} x_{1,0}\omega_0 + x_{1,1}\omega_1 + \cdots + x_{1,d}\omega_d \\ x_{2,0}\omega_0 + x_{2,1}\omega_1 + \cdots + x_{2,d}\omega_d \\ \vdots \\ x_{N,0}\omega_0 + x_{N,1}\omega_1 + \cdots + x_{N,d}\omega_d \end{bmatrix}$$

$$X\vec{\omega} - \vec{y} = X\vec{\omega} - \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} = \begin{bmatrix} x_{1,0}\omega_0 + x_{1,1}\omega_1 + \cdots + x_{1,d}\omega_d - y_1 \\ x_{2,0}\omega_0 + x_{2,1}\omega_1 + \cdots + x_{2,d}\omega_d - y_2 \\ \vdots \\ x_{N,0}\omega_0 + x_{N,1}\omega_1 + \cdots + x_{N,d}\omega_d - y_N \end{bmatrix}$$

- Reescrevendo...

$$X\vec{\omega} - \vec{y} = \begin{bmatrix} x_{1,0}\omega_0 + x_{1,1}\omega_1 + \cdots + x_{1,d}\omega_d - y_1 \\ x_{2,0}\omega_0 + x_{2,1}\omega_1 + \cdots + x_{2,d}\omega_d - y_2 \\ \vdots \\ x_{N,0}\omega_0 + x_{N,1}\omega_1 + \cdots + x_{N,d}\omega_d - y_N \end{bmatrix} = \begin{bmatrix} \sum_{j=0}^d (x_{1,j}\omega_j) - y_1 \\ \sum_{j=0}^d (x_{2,j}\omega_j) - y_2 \\ \vdots \\ \sum_{j=0}^d (x_{N,j}\omega_j) - y_N \end{bmatrix}$$

Regressão Linear - Medindo o erro

$$|X\vec{\omega} - \vec{y}|^2 = \left(\sum_{j=0}^d (x_{1,j}\omega_j) - y_1 \right)^2 + \cdots + \left(\sum_{j=0}^d (x_{N,j}\omega_j) - y_N \right)^2$$

$$\Rightarrow |X\vec{\omega} - \vec{y}|^2 = \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j}\omega_j) - y_i \right)^2$$

$$E(\vec{\omega}) = \frac{1}{N} |X\vec{\omega} - \vec{y}|^2 = \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j}\omega_j) - y_i \right)^2$$

$$\text{como } h(\vec{x}_i) = \sum_{j=0}^d \omega_j x_{i,j} \Rightarrow E(\vec{\omega}) = \frac{1}{N} \sum_{i=1}^N (h(\vec{x}_i) - y_i)^2$$

Equação Geral

$$y = X\vec{w} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

onde $y \in \mathbb{R}^n$, $X \in \mathbb{R}^{n \times p}$ e $\vec{w} \in \mathbb{R}^p$.

Objetivo

Estimar $\vec{w}$ minimizando o erro quadrático:

$$\min_{\vec{w}} \|X\vec{w} - y\|_2^2$$

- Para minimizar $E(\vec{\omega})$, tomamos $\nabla E(\vec{\omega}) = \vec{0}$
 - Recorde que $\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
- A questão é: minimizar em relação a que?

- Para minimizar $E(\vec{\omega})$, tomamos $\nabla E(\vec{\omega}) = \vec{0}$
 - Recorde que $\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
- A questão é: minimizar em relação a que?
 - Não queremos o ponto $(x_0, \dots, x_d)$ (ou seja, atributos) em que o erro é mínimo, mas sim o conjunto de pesos $(\omega_0, \dots, \omega_d)$ que minimiza o erro

- Para minimizar $E(\vec{\omega})$, tomamos $\nabla E(\vec{\omega}) = \vec{0}$
 - Recorde que $\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$
- A questão é: minimizar em relação a que?
 - Não queremos o ponto $(x_0, \dots, x_d)$ (ou seja, atributos) em que o erro é mínimo, mas sim o conjunto de pesos $(\omega_0, \dots, \omega_d)$ que minimiza o erro
 - Então...

$$\nabla E(\vec{\omega}) = \left(\frac{\partial E(\vec{\omega})}{\partial \omega_0}, \frac{\partial E(\vec{\omega})}{\partial \omega_1}, \dots, \frac{\partial E(\vec{\omega})}{\partial \omega_d} \right)$$

- Como $E(\vec{\omega}) = \frac{1}{N} |X\vec{\omega} - \vec{y}|^2$, X e $\vec{y}$ tornam-se constantes
 - São os dados (dados de entrada e saída) que alguém nos deu
 - O parâmetro a ser manipulados é ω
- Então...

$$\nabla E(\vec{\omega}) = \frac{2}{N} X^t (X\vec{\omega} - \vec{y})$$

- Novamente...

$$E(\vec{\omega}) = \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j} \omega_j) - y_i \right)^2$$

$$\Rightarrow \nabla E(\vec{\omega}) = \begin{bmatrix} \frac{\partial E(\vec{\omega})}{\partial \omega_0} \\ \vdots \\ \frac{\partial E(\vec{\omega})}{\partial \omega_d} \end{bmatrix} = \begin{bmatrix} \frac{2}{N} \sum_{i=1}^N \left[x_{i,0} \left(\sum_{j=0}^d (x_{i,j} \omega_j) - y_i \right) \right] \\ \vdots \\ \frac{2}{N} \sum_{i=1}^N \left[x_{i,d} \left(\sum_{j=0}^d (x_{i,j} \omega_j) - y_i \right) \right] \end{bmatrix}$$

- Olhando para $X^t(X\vec{\omega} - \vec{y})$ (slide 22):

$$X\vec{\omega} - \vec{y} = \begin{bmatrix} x_{1,0}\omega_0 + x_{1,1}\omega_1 + \cdots + x_{1,d}\omega_d - y_1 \\ x_{2,0}\omega_0 + x_{2,1}\omega_1 + \cdots + x_{2,d}\omega_d - y_2 \\ \vdots \\ x_{N,0}\omega_0 + x_{N,1}\omega_1 + \cdots + x_{N,d}\omega_d - y_N \end{bmatrix}$$

$$\Rightarrow X\vec{\omega} - \vec{y} = \begin{bmatrix} \sum_{j=0}^d x_{1,j}\omega_j - y_1 \\ \vdots \\ \sum_{j=0}^d x_{N,j}\omega_j - y_N \end{bmatrix}$$

- Então:

$$X^t(X\vec{\omega} - \vec{y}) = \begin{bmatrix} X_{1,0} & X_{2,0} & \cdots & X_{N,0} \\ X_{1,1} & X_{2,1} & \cdots & X_{N,1} \\ \vdots & \vdots & \ddots & \vdots \\ X_{1,d} & X_{2,d} & \cdots & X_{N,d} \end{bmatrix} \begin{bmatrix} \sum_{j=0}^d X_{1,j}\omega_j - y_1 \\ \sum_{j=0}^d X_{2,j}\omega_j - y_2 \\ \vdots \\ \sum_{j=0}^d X_{N,j}\omega_j - y_N \end{bmatrix}$$

- $\frac{2}{N} X^t (X\vec{\omega} - \vec{y}) =$

$$\begin{bmatrix} \frac{2}{N} \left[x_{1,0} \left(\sum_{j=0}^d x_{1,j} \omega_j - y_1 \right) \right] + \cdots + \frac{2}{N} \left[x_{N,0} \left(\sum_{j=0}^d x_{N,j} \omega_j - y_N \right) \right] \\ \vdots \\ \frac{2}{N} \left[x_{1,d} \left(\sum_{j=0}^d x_{1,j} \omega_j - y_1 \right) \right] + \cdots + \frac{2}{N} \left[x_{N,d} \left(\sum_{j=0}^d x_{N,j} \omega_j - y_N \right) \right] \end{bmatrix}$$

Que corresponde a $\nabla E(\vec{\omega})$, visto no slide 27

- Então, para minimizar o erro, temos que

$$\nabla E(\vec{\omega}) = \frac{2}{N} X^t (X\vec{\omega} - \vec{y}) = \vec{0}$$

$$\Rightarrow X^t (X\vec{\omega} - \vec{y}) = \vec{0}$$

$$\Rightarrow X^t X \vec{\omega} = X^t \vec{y}$$

$$\text{e } \vec{\omega} = (X^t X)^{-1} X^t \vec{y} = X^\dagger \vec{y}, \text{ onde } X^\dagger = (X^t X)^{-1} X^t$$

$X^\dagger$ é a pseudo-inversa de X

- Vale lembrar que $X^t X$ não necessariamente é inversível
 - $X^t X$ é quadrada, o que ajuda
 - Para ser inversível, $A^{-1} A = I$ (matriz identidade)
- Se X fosse quadrada e inversível, poderíamos fazer $X\vec{\omega} - \vec{y} = 0$
 - Bastaria multiplicar ambos os lados por X^{-1} e teríamos $\vec{\omega} = X^{-1}\vec{y}$
 - Mas X ser quadrada implicaria termos o mesmo número de exemplos e atributos
 - Normalmente temos poucos atributos e muitos exemplos

- Ainda assim, $X^\dagger$ é computacionalmente interessante:

$$\left(\underbrace{\begin{bmatrix} \vdots & \vdots \end{bmatrix}}_{d+1 \times N} \underbrace{\begin{bmatrix} \vdots \\ \vdots \end{bmatrix}}_{N \times d+1} \right)^{-1} \underbrace{\begin{bmatrix} \vdots \\ \vdots \end{bmatrix}}_{d+1 \times N} \Rightarrow \underbrace{\left(\underbrace{\begin{bmatrix} \vdots & \vdots \end{bmatrix}}_{d+1 \times d+1} \right)^{-1} \underbrace{\begin{bmatrix} \vdots \\ \vdots \end{bmatrix}}_{d+1 \times N}}_{d+1 \times N}$$

Como em geral $N \gg d$, isso torna-se bem atraente

Regressão Linear - Algoritmo

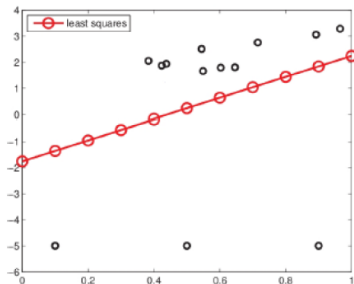
- Construa a matriz X e o vetor $\vec{y}$ a partir do conjunto de dados $(x_1, y_1), \dots, (x_N, y_N)$ como segue:

$$\underbrace{X = \begin{bmatrix} x_{1,0} & x_{1,1} & \cdots & x_{1,d} \\ x_{2,0} & x_{2,1} & \cdots & x_{2,d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{N,0} & x_{N,1} & \cdots & x_{N,d} \end{bmatrix}}_{\text{matriz de dados de entrada}}, \quad \underbrace{\vec{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix}}_{\text{vetor-alvo}}$$

- Calcule a pseudo-inversa $X^\dagger = (X^t X)^{-1} X^t$
- Retorne $\vec{\omega} = X^\dagger \vec{y}$

Regressão Linear - Limitações

- Se houver anomalias (outliers) nos dados, isso pode resultar em um ajuste ruim:

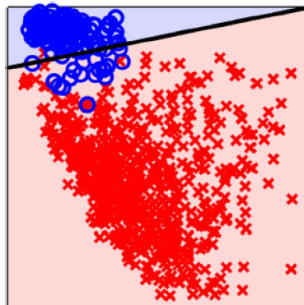


- Isso se deve ao fato do erro penalizar desvios quadraticamente $\rightarrow$ pontos longe da reta a afetam mais que pontos próximos

- Prós:
 - Resultados de fácil interpretação
 - Computacionalmente barato
 - Trabalha tanto com valores numéricos quanto nominais
 - Basta mapear as categorias a valores. Ex: $c_1 = +1$ e $c_2 = -1$ (note que $\pm 1 \in \mathbb{R}$)
 - Use a regressão linear para obter $\vec{\omega}$, onde $\vec{\omega} \cdot \vec{x}_i \approx y_i = \pm 1$
 - Estando próximo de ± 1 , $\text{sin}(\vec{\omega} \cdot \vec{x})$ estará correto
- Contras:
 - Modela de maneira pobre dados não lineares

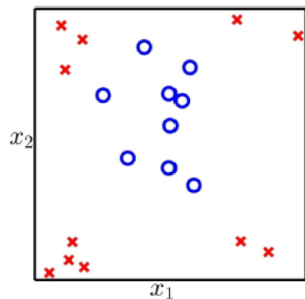
Regressão Linear para classificação

- Problemas no uso como classificação:
 - Tentará fazer todos os pontos ± 1 e não conseguirá
 - Considera -2, -3 etc como erros, quando estariam corretos na classificação
 - Vai minimizar o erro
 - Ainda assim, boa aproximação inicial



Transformação não Linear

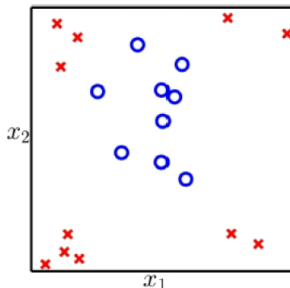
- Considere o seguinte conjunto de dados
 - São linearmente separáveis?
 - Linearmente em que?



Transformação não Linear

- Considere o seguinte conjunto de dados
 - São linearmente separáveis?
 - Linearmente em que?
- A regressão implementa

$$\sum_{i=0}^d \omega_i x_i$$



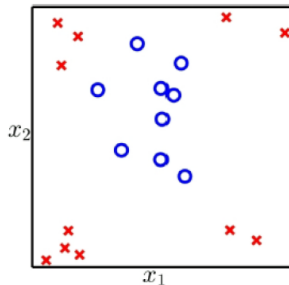
Transformação não Linear

- Considere o seguinte conjunto de dados
 - São linearmente separáveis?
 - Linearmente em que?

- A regressão implementa

$$\sum_{i=0}^d \omega_i x_i$$

- Basta tomarmos os dados como lineares em relação aos pesos, não a x_i

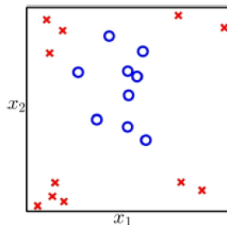


Transformação não Linear

- Como?

- Repare que se colocarmos a origem no centro do gráfico, os azuis estarão mais perto dela e os vermelhos mais longe
- Basta então dar mais peso à distância ao centro, mudando o sistema de coordenadas:

$$(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$$



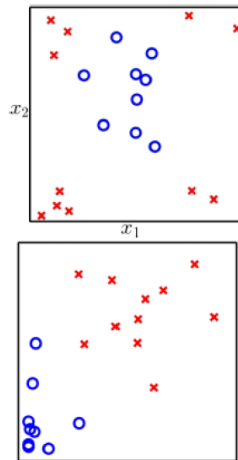
Transformação não Linear

- Como?

- Repare que se colocarmos a origem no centro do gráfico, os azuis estarão mais perto dela e os vermelhos mais longe
- Basta então dar mais peso à distância ao centro, mudando o sistema de coordenadas:

$$(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$$

- E o novo gráfico, com essa transformação, fica:



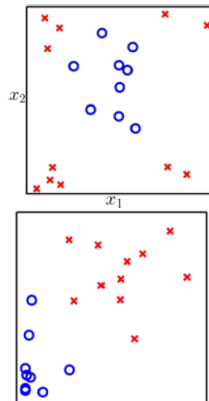
Transformação não Linear

- Como?

- Repare que se colocarmos a origem no centro do gráfico, os azuis estarão mais perto dela e os vermelhos mais longe
- Basta então dar mais peso à distância ao centro, mudando o sistema de coordenadas:

$$(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$$

- E o novo gráfico, com essa transformação, fica:
 - Linearmente separáveis



Modelos Lineares Kernelizados

- O modelo linear clássico assume:

$$f(\vec{x}_i) = \vec{w}^\top \vec{x}_i$$

- Quando os dados não são linearmente separáveis, podemos mapeá-los para um espaço de características de maior dimensão:

$$\phi : \mathbb{R}^p \rightarrow \mathcal{H}, \quad \vec{x}_i \mapsto \phi(\vec{x}_i)$$

- No espaço transformado, o modelo torna-se linear em $\phi(\vec{x}_i)$:

$$f(\vec{x}_i) = \vec{w}^\top \phi(\vec{x}_i)$$

Formulação no Espaço de Características

- Seja $\Phi = [\phi(x_1)^\top; \phi(x_2)^\top; \dots; \phi(x_n)^\top]$ a matriz de características transformadas.
- O modelo linear pode ser escrito como:

$$y = \Phi\beta + \varepsilon$$

- A solução de mínimos quadrados é formalmente a mesma:

$$\vec{\beta} = (\Phi^\top \Phi)^{-1} \Phi^\top y$$

- No entanto, Φ pode ter dimensão muito alta ou até infinita.

- Alta dimensão ou espaço infinito:

$$\beta = \Phi^\top \alpha, \quad K = \Phi \Phi^\top$$

- Mínimos quadrados: $y = K\alpha \Rightarrow \hat{\alpha} = K^{-1}y$
- Predição novo ponto:

$$\hat{y}_{\text{novo}} = \sum_i \alpha_i K(x_i, x_{\text{novo}})$$

Predição substituindo $\hat{\beta}$

Partindo de $\hat{\beta} = (X^T X)^{-1} X^T y$ (assumindo $X^T X$ inversível), a predição para x é

$$\hat{f}(x) = x^T \hat{\beta} = x^T (X^T X)^{-1} X^T y.$$

Definindo $h(x) = X(X^T X)^{-1} x$, obtemos

$$\hat{f}(x) = h(x)^T y = \sum_{i=1}^n h_i(x) y_i.$$

Para os pontos de treino (todas as predições) usamos a matriz *hat*:

Se $X^T X$ for singular, substitua a inversa por Moore–Penrose X^+ :

$$\hat{\beta} = X^+ y, \quad \hat{f}(x) = x^T X^+ y.$$

Na versão kernelizada (com Φ e $K = \Phi \Phi^T$):

$$\hat{f}(x) = \phi(x)^T (\Phi^T \Phi)^{-1} \Phi^T y = k_x^T K^{-1} y = \sum_{i=1}^n \alpha_i K(x_i, x).$$

Modelo Linear Kernelizado

- No espaço transformado, o modelo torna-se linear em $\phi(\vec{x}_i)$:

$$f(\vec{x}_i) = \phi(x_i)^\top \vec{w}$$

- A solução de mínimos quadrados é formalmente a mesma:

$$\vec{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y$$

- Logo, podemos escrever

$$f(\vec{x}_i) = (\phi(\vec{x}_i)^\top (\Phi^\top \Phi)^{-1} \Phi^\top y$$

$$f(\vec{x}_i) = \phi(x_i)^\top (\Phi^\top \Phi)^{-1} \Phi^\top y$$

Modelo Linear Kernelizado

- Dado $K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$, podemos escrever $K = \Phi \Phi^\top$ (matriz Gram)
- Pode-se mostrar que $\Phi^\top K^{-1} = (\Phi^\top \Phi)^{-1} \Phi^\top$
- Logo

$$k_x = [K(x_1, x); K(x_2, x); \dots; K(x_n, x)]$$

$$k_x = [\phi(x_1)^\top \phi(x); \phi(x_2)^\top \phi(x); \dots; \phi(x_n)^\top \phi(x)]$$

$$k_x = [\phi(x_1)^\top; \phi(x_2)^\top; \dots; \phi(x_n)^\top] \phi(x)$$

$$k_x = \Phi \phi(x)$$

- Portanto $k_x^\top = (\Phi \phi(x))^\top = \phi(x)^\top \Phi^\top$
- A saída

$$f(\vec{x}_i) = \phi(x_i)^\top (\Phi^\top \Phi)^{-1} \Phi^\top y$$

- Reescrevendo a saída

$$f(\vec{x}_i) = \phi(x_i)^\top \Phi^\top K^{-1} y$$

Modelo Linear Kernelizado

- Logo

$$f(\vec{x}_i) = \phi(x_i)^\top \Phi^\top K^{-1} y$$

- Logo

$$f(\vec{x}_i) = k_x^\top K^{-1} y$$

- Definido $\alpha = K^{-1} y$

- temos que

$$f(\vec{x}_i) = k_x^\top \alpha$$

- logo

$$f(\vec{x}_i) = \alpha^\top k_x$$

- Reescrevendo

$$f(\vec{x}_i) = \sum_{i=1}^n \alpha_i K(x_i, x)$$

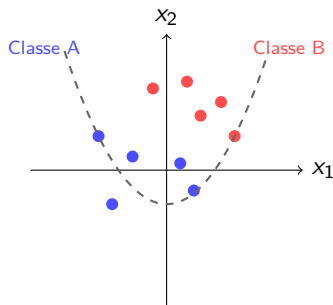
Kernels Mais Comuns

Tipo	Fórmula	Observação
Linear	$x_i^\top x_j$	Reduz ao modelo linear
Polinomial	$(x_i^\top x_j + c)^d$	Combinações polinomiais
RBF/Gaussiano	$\exp(-\ x_i - x_j\ ^2 / 2\sigma^2)$	Espaço infinito dimensional
Sigmoide	$\tanh(\kappa x_i^\top x_j + \theta)$	Redes neurais kernelizadas

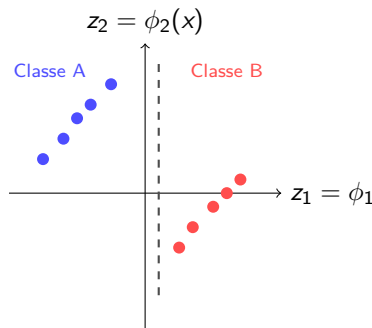
- Aprendizagem depende apenas de produtos internos $K(x_i, x_j)$
- Mantemos estrutura linear, capturando não-linearidades
- Predição: $\hat{y} = \sum_i \alpha_i K(x_i, x_{\text{novo}})$

Modelo Linear vs. Modelo Kernelizado

Espaço Original $\mathbb{R}^2$



Espaço de Características $\mathcal{H}$



No espaço original, a fronteira é **não linear**. Após a transformação $\phi(x)$, os dados tornam-se **linearmente separáveis**:

$$K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$$

- Coleta dos dados
- Preparação
 - Se numéricos, ok, se nominais, mapeie-os a valores numéricos
- Treinamento
 - Encontre os pesos da regressão
- Teste
 - Meça a correlação entre os valores previstos e os dados reais, para medir o sucesso do modelo

Modelo linear: $\hat{y} = \mathbf{X}\beta$

Estimação por mínimos quadrados:

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$$

Solução: $\beta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

Modelo linear: $\hat{y} = \mathbf{X}\beta$

Estimação por mínimos quadrados:

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$$

Solução: $\beta = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

- **Ridge (Norma 2):** penaliza $\|\beta\|_2^2$
- **Lasso (Norma 1):** penaliza $\|\beta\|_1$
- **Elastic Net:** combinação das duas normas

$$\min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda_1 \|\beta\|_1 + \lambda_2 \|\beta\|_2^2$$

Modelos Lineares Regularizados

Quadrados Mínimos Regularizados

- A fim de controlar o sobre-ajuste, a função erro a ser minimizada pode ser definida como

$$E(\vec{\omega}) + \gamma E_{\vec{\omega}}(\vec{\omega})$$

onde γ é coeficiente de regularização que controla a importância relativa do erro dependente dos dados e o termo de regularização $E_{\vec{\omega}}(\vec{\omega})$.

- Uma das formas mais simples para termo de regularização é dada pela equação abaixo

$$E_{\vec{\omega}}(\vec{\omega}) = \vec{\omega}^t \vec{\omega}$$

- Considerando a função erro como a soma dos quadrados do erro dada por

$$E(\vec{\omega}) = \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j} \omega_j) - y_i \right)^2$$

Quadrados Mínimos Regularizados

- A função erro total é definida como

$$\frac{1}{N} \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j} \omega_j) - y_i \right)^2 + \gamma \vec{\omega}^T \vec{\omega}$$

- Esta escolha particular para o termo de regularização é conhecida na literatura de Aprendizado de Máquina como decaimento dos pesos por que em algoritmos de aprendizado sequencial, este encoraja os valores dos pesos cair em direção a zero.
- Em estatística, este fornece um exemplo de um método de encolhimento de parâmetro, por que este reduz os valores dos parâmetros em direção a zero

Quadrados Mínimos Regularizados

- Tomando o gradiente com relação a w , fazendo igual a zero e isolando w , nós obtemos

$$\tilde{w} = \left(\frac{\gamma}{N} I + X^t X \right)^{-1} X^t \vec{y}$$

- Este representa uma extensão da solução do Quadrados Mínimos
- Um termo de regularização mais geral é algumas vezes usada, para o qual o erro de regularizado assume a seguinte forma

$$\frac{1}{N} \sum_{i=1}^N \left(\sum_{j=0}^d (x_{i,j} w_j) - y_i \right)^2 + \gamma \sum_{j=1}^M |w_j|^q$$

- $q = 2$ corresponde para um regularizador quadrático
- $q = 1$ é conhecido como *lasso* na literatura de estatística.

Motivação para Regularização

- Em problemas com alta dimensionalidade ($p > n$), o estimador por mínimos quadrados se torna instável.
- A regularização reduz a variância ao custo de introduzir viés.
- Também é usada para seleção de variáveis e evitar overfitting.

Ridge Regression (Tikhonov)

É possível adicionar um termo de regularização, que penaliza o crescimento da norma do vetor w , produzindo o problema regularizado de regressão de quadrados mínimos, também denominado de ridge regression, na forma

$$\hat{w}^{ridge} = \arg \min_w \{ \|Y - Xw\|^2 + \lambda \|w\|^2 \}$$

Ridge Regression (Tikhonov)

É possível adicionar um termo de regularização, que penaliza o crescimento da norma do vetor w , produzindo o problema regularizado de regressão de quadrados mínimos, também denominado de ridge regression, na forma

$$\hat{w}^{ridge} = \arg \min_w \{ \|Y - Xw\|^2 + \lambda \|w\|^2 \}$$

- Penaliza o tamanho dos coeficientes (ℓ_2).
- Não realiza seleção de variáveis.
- Solução fechada: $\hat{W} = (X^T X + \lambda I)^{-1} X^T Y$

Ridge Regression (Tikhonov)

- A definição de um valor para o parâmetro de regularização $\lambda \geq 0$ pode ser feita por técnicas de valização cruzada
- O modelo regularizado de regressão linear assume então a forma

$$w^T x = y^T X(X^T X + \lambda I)^{-1} x$$

- Podemos agora usar a equivalência $X(X^T X + \lambda I)^{-1} = (X X^T + \lambda I)^{-1} X$ de modo a obter o modelo de regressão em sua forma dual:

$$w^T x = y^T (X X^T + \lambda I)^{-1} X x$$

- Já sabemos que, para $N > n$, a forma dual é mais custosa computacionalmente. Mas esta forma dual pode ser representada de modo a conduzir a um importante resultado que sustenta a regressão por função kernel.

Pseudocódigo - Quadrados Minimos Regularizado (Ridge Regression)

Algorithm Ajuste de Ridge Regression

Entrada: matriz X , vetor y , parâmetro de regularização λ

Adicione coluna de 1s em X (intercepto)

Calcule $A \leftarrow X^T X + \lambda I$

Calcule $b \leftarrow X^T y$

Resolva $Aw = b$

Saída: vetor de coeficientes w

Lasso (Least Absolute Shrinkage)

- Uma formulação alternativa considera a penalização com a norma ℓ_1 , produzindo

$$\hat{w}^{lasso} = \arg \min_w \{ \|Y - Xw\|^2 + \lambda \|w\|_1 \}$$

- A vantagem do LASSO esta principalmente no fato de que múltiplos termos do vetor w acabam sendo anulados, o que representa um processo de seleção de variáveis e leva a modelos de aproximação mais parcimoniosos
- Penaliza a soma dos valores absolutos dos coeficientes (ℓ_1).
- Pode zerar coeficientes $\Rightarrow$ faz seleção de variáveis.
- Solução encontrada via algoritmos iterativos (ex: LARS, coordinate descent).

- A motivação geométrica que permite justificar a anulação de um subconjunto de parâmetros ajustáveis é apresentada na figura a seguir.
- Nesta figura, apresentam-se os lugares geométricos de valores constantes para $J(w)$ e $\|w\|_1$

Pseudocódigo - Lasso (Coordinate Descent)

Algorithm Ajuste de Lasso via Coordinate Descent

Entrada: matriz X , vetor y , parâmetro λ

Inicialize $w \leftarrow 0$

repeat

for cada coeficiente β_j **do**

 Calcule o resíduo parcial: $r_j \leftarrow y - X_{-j}w_{-j}$

 Calcule $z_j \leftarrow X_j^T r_j$

 Atualize $w_j \leftarrow \text{sign}(z_j) \cdot \max(|z_j| - \lambda, 0)$

end for

until convergência

Saída: vetor de coeficientes w

$$\hat{w}^{enet} = \arg \min_w \{ \|Y - Xw\|^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|^2 \}$$

- Combina Ridge e Lasso
- Controla suavização e seleção simultaneamente

Algorithm Ajuste de Elastic Net via Coordinate Descent

Entrada: X , y , parâmetros λ_1 , λ_2

Inicialize $w \leftarrow 0$

repeat

for cada coeficiente w_j **do**

$r_j \leftarrow y - X_{-j}w_{-j}$

$z_j \leftarrow X_j^T r_j$

$s_j \leftarrow \lambda_1$

$\beta_j \leftarrow \frac{\text{sign}(z_j) \cdot \max(|z_j| - s_j, 0)}{1 + \lambda_2}$

end for

until convergência

Saída: vetor w

Encontrar o gradiente da Elastic Net

Implementar métodos de otimização

Conjunto de dados diabetes - UCI

Código Python - Ridge Regression

```
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=100, n_features=10, noise=10,
                      random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
                                                    =42)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
print("Ridge Regression coef_")
```

Código Python - Lasso

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=100, n_features=10, noise=10,
                      random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
                                                    =42)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
print("Lasso_coeficientes:", lasso.coef_)
```

Código Python - Elastic Net

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=100, n_features=10, noise=10,
                      random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state
                                                    =42)

elastic_net = ElasticNet(alpha=0.1, l1_ratio=0.5)  # l1_ratio controla
mistura entre L1 e L2
elastic_net.fit(X_train, y_train)
print("Elastic_Net_coeficientes:", elastic_net.coef_)
```

Regressão Logística

Regressão Logística Multiplas Classes

O que é Regressão Logística?

- Modelo probabilístico para classificação.
- Mapeia entradas para probabilidades de classes através de uma função logística / softmax.
- Amplamente usado por ser simples, interpretável e eficiente.

Linear vs Logístico

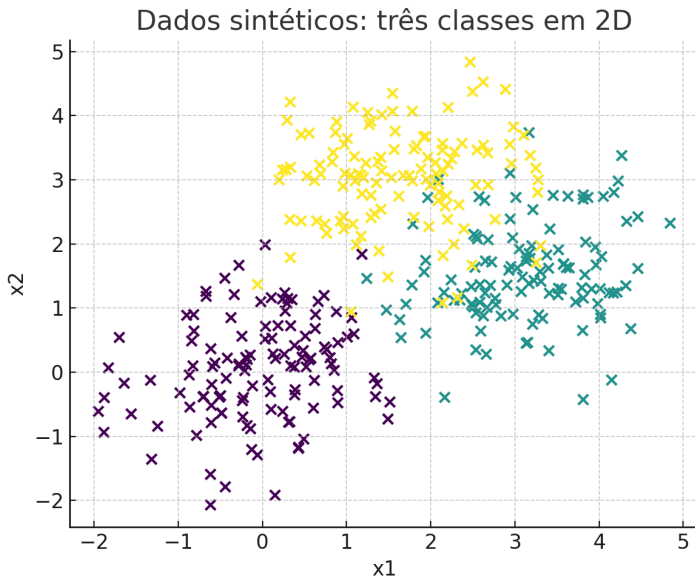
- Regressão linear prevê valores numéricos; regressão logística prevê probabilidades de classe.
- Para múltiplas classes, usamos **softmax** em vez da função sigmoide.

Motivação para multiclasse

- Muitas aplicações têm mais de duas categorias (ex.: reconhecimento de dígitos, classificação de texto).
- Estratégias: *one-vs-rest*, *softmax (multinomial)* — aqui estudamos softmax diretamente.

- Saúde, detecção de doenças; NLP; visão computacional; econometria; classificação de clientes.

Exemplo ilustrativo



Softmax: definição

Dados vetores de pontuação $z_k = w_k^\top x$, definimos:

$$p(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=1}^K \exp(z_j)}$$

Interpretação: probabilidade normalizada sobre as classes.

Entropia cruzada (loss)

Para um exemplo (x_i, y_i) a perda cross-entropy é:

$$\ell_i = - \sum_{k=1}^K \mathbb{1}_{\{y_i = k\}} \log p(y = k \mid x_i)$$

Loss total média:

$$L = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log p_{ik}$$

Log-likelihood e regularização

Maximizar a log-likelihood é equivalente a minimizar a entropia cruzada. Podemos adicionar termo de regularização $\frac{\lambda}{2} \|W\|^2$ para controlar overfitting.

Gradiente (derivação)

Com X ($n \times d$), W ($d \times K$) e Y one-hot:

$$\nabla_W L = \frac{1}{n} X^\top (P - Y) + \lambda W$$

onde P é a matriz de probabilidades ($n \times K$).

Hessiana (estrutura)

A Hessiana completa é grande: bloco-estrutura em termos de matrizes $S_i = \text{diag}(p_i) - p_i p_i^\top$:

$$H = \frac{1}{n} \sum_{i=1}^n X_i^\top S_i X_i + \lambda I$$

(Escrever a forma completa leva a uma matriz $dK \times dK$.)

A perda cross-entropy para softmax é convexa em relação a parâmetros lineares W (quando considerada sem restrições de simetria), facilitando otimização global.

Comparação de métodos de otimização

- Gradiente Descendente: simples, escala para grandes bases.
- Newton: convergência rápida (quadrática) mas custo alto por iteração (Hessiana).
- SGD / mini-batch: boa prática para grandes datasets.

Gradiente Descendente (GD)

Iteração básica:

$$\mathcal{W}^{(t+1)} = \mathcal{W}^{(t)} - \eta \nabla_{\mathcal{W}} L(\mathcal{W}^{(t)})$$

Discussão de taxa de aprendizado e normalização dos dados.

Algoritmo iterativo (passo a passo)

- 1 Inicializar W (zeros ou pequena ruído)
- 2 Calcular scores $S = XW$
- 3 Calcular probabilidades $P = \text{softmax}(S)$
- 4 Calcular gradiente e atualizar W
- 5 Repetir até convergência

Vantagens: velocidade por iteração, eficiência em memória. Ajustes de learning rate, momentum, ADAM podem ser aplicados.

Método de Newton para softmax

Sistema linear para atualização:

$$\Delta = H^{-1} \nabla_W L$$

Atualização: $W \leftarrow W - \Delta$. Discussão do custo computacional.

Vantagens x Desvantagens

- Newton: menos iterações, custo alto em cada iteração.
- GD/SGD: fáceis, escaláveis, dependem de hiperparâmetros.

Critérios de convergência

Mudar tolerância em $\|\nabla L\|$, mudança relativa na loss, número máximo de iterações.

Pseudocódigo: Softmax (GD)

```
#  $X: n \times d$ ,  $y: n$   
W = zeros(d, K)  
for t in range(max_iter):  
    scores = X.dot(W)  
    probs = softmax(scores)  
    grad = X.T.dot(probs - Y) / n + reg * W  
    W -= lr * grad
```

Pseudocódigo: Método de Newton (esquemático)

```
# compute full Hessian H (dK x dK)  
g = grad_vectorized           # vectorized gradient (dK)  
delta = solve(H, g)  
W_vector -= delta
```

Discussão sobre $O(nd^2K)$ e custo de armazenar H ($dK \times dK$) para Newton na prática.

Importações e preparação dos dados

```
import numpy as np
# carregar/gerar dados
# adicionar termo de bias:  $X = np.hstack([np.ones((n,1)),$   
       $X_{raw}])$ 
```

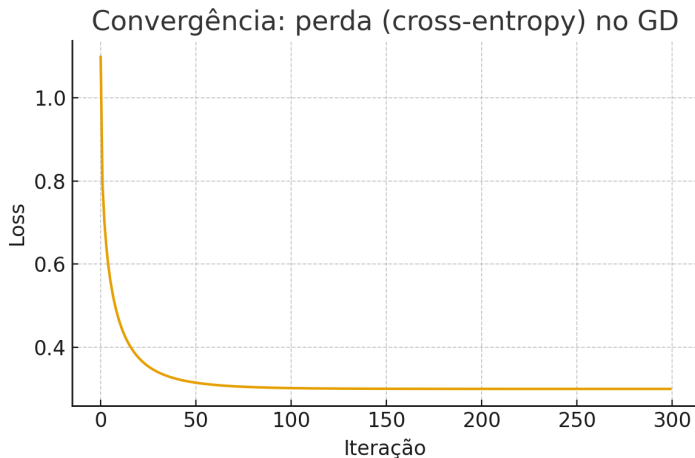
Função softmax (implementação)

```
def softmax(z):  
    z = z - np.max(z, axis=1, keepdims=True)  
    exp_z = np.exp(z)  
    return exp_z / np.sum(exp_z, axis=1, keepdims=True)
```

Cálculo de perda, gradiente e Hessiana (trechos)

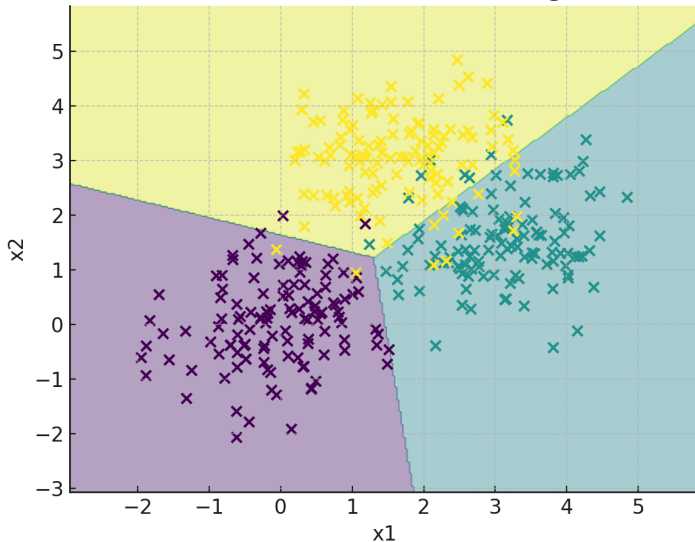
```
# loss
loss = -np.sum(Y * np.log(probs + 1e-12)) / n + 0.5*reg*
      np.sum(W*W)

# grad
grad = X.T.dot(probs - Y) / n + reg*W
```



Fronteira de decisão (exemplo)

Fronteira de decisão (Softmax Regression)



Código completo (arquivo em /codigo)

```
# veja codigo/data_and_plots.py e codigo/softmax_impl.py  
# os scripts geram as figuras e implementam o treinamento  
em NumPy
```

Pontos Fortes e Limitações

- Forte baseline interpretável e eficiente para problemas lineares.
- Limitações: incapacidade de capturar relações não-lineares sem features adicionais.

- Regularização L1/L2, features polinomiais, kernel methods, redes neurais.
- Otimização avançada: Adam, L-BFGS, minibatch com momentum.

- slides.tex (Beamer)
- figuras/ (PNG geradas)
- codigo/ (scripts Python)

Obrigado!

Perguntas?

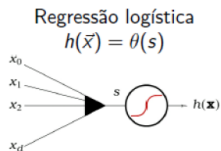
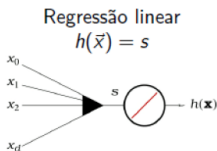
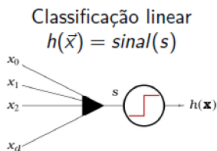
Obrigado!
Perguntas e Discussão

Regressão Logística

No modelo linear combinamos linearmente a entrada com os pesos:

$$s = \sum_{i=0}^d \omega_i x_i$$

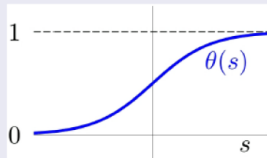
E então fazemos algo com o resultado, que pode ser:



A Função Logística

Pegamos s e aplicamos uma não-linearidade θ a ele $\rightarrow$ a Função Logística:

$$\theta(s) = \frac{e^s}{1 + e^s}$$

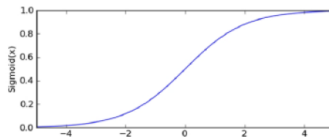


Em regressão logística a saída $h(\vec{x}) = \theta(s)$ é interpretada como uma probabilidade

- Varia de 0 a 1. Se s muito negativo, $\theta(s)$ aproxima-se de 0, se muito positivo, de 1
- Há um *limiar suave* (soft threshold)

Regressão Logística

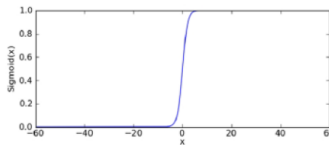
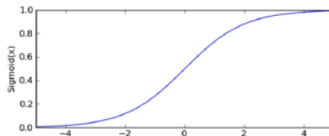
Ainda que a função logística pareça distante da função degrau usada na classificação ...



Regressão Logística

Ainda que a função logística pareça distante da função degrau usada na classificação ...

Em uma escala grande o suficiente, a sigmóide parece com a função degrau



Exemplo

Previsão de ataques cardíacos em dado intervalo de tempo:

- Entrada $\vec{x}$: nível de colesterol, idade, peso etc
- Saída $\theta(s)$: probabilidade de um ataque cardíaco
- Lembrando que o sinal $s = \vec{\omega} \cdot \vec{x} \rightarrow$ fator de risco
 - Note que s permanece linear...

Modelo mais interessante (e realista) que uma simples classificação, em que a resposta seria que terá (ou não) o ataque $\rightarrow$ 100% de certeza

Suponha que temos um conjunto de dados $(\vec{x}, y)$, com y binário, gerado por uma função-alvo com ruído:

$$P(y|\vec{x}) = \begin{cases} f(\vec{x}) & \text{para } y = +1 \\ 1 - f(\vec{x}) & \text{para } y = -1 \end{cases}$$

Junto a cada ponto $(\vec{x})$, é dada uma saída (y) que é afetada por uma probabilidade

- Probabilidade de ataque cardíaco ($y = +1$) ou não ($y = -1$), dado os dados

$$P(y|\vec{x}) = \begin{cases} f(\vec{x}) & \text{para } y = +1 \\ 1 - f(\vec{x}) & \text{para } y = -1 \end{cases}$$

Estamos tentando aprender a função-alvo $f : \mathbb{R}^d \rightarrow [0, 1]$

- A probabilidade de um ataque

Queremos então aprender $g(\vec{x}) = \theta(\vec{\omega} \cdot \vec{x}) \approx f(\vec{x})$

- A hipótese final que aproxima f

A questão é então como escolho os pesos (parâmetros) de modo a que a hipótese da regressão logística reflita a função-alvo

Nesse modelo, para cada $(\vec{x}, y)$, y é gerado pela probabilidade $f(\vec{x})$

- Avaliamos diferentes hipóteses conforme a probabilidade de cada uma delas ser realmente a função-alvo que gerou os dados

Assumamos que uma determinada hipótese represente a função-alvo (ou seja, $h = f$)

- Qual a probabilidade de gerar esses dados se essa suposição for verdadeira?
- Ou, se $h = f$, qual a probabilidade de obtermos y a partir de $\vec{x}$?
- Note que há uma inversão: não queremos saber a probabilidade da hipótese dados os dados, mas sim a probabilidade dos dados dada a hipótese

Olhemos então a distribuição de probabilidade que assumimos:

$$P(y|\vec{x}) = \begin{cases} f(\vec{x}) & \text{para } y = +1 \\ 1 - f(\vec{x}) & \text{para } y = -1 \end{cases}$$

Sob a suposição de que $h = f$, essa probabilidade torna-se

$$P(y|\vec{x}) = \begin{cases} h(\vec{x}) & \text{para } y = +1 \\ 1 - h(\vec{x}) & \text{para } y = -1 \end{cases}$$

$$P(y|\vec{x}) = \begin{cases} h(\vec{x}) & \text{para } y = +1 \\ 1 - h(\vec{x}) & \text{para } y = -1 \end{cases}$$

Alguns pontos são interessantes:

- $P(y|\vec{x})$ trata de somente um ponto ($\vec{x}$)
- Não é interessante que a fórmula trate de casos (descontinuidade) – queremos algo mais analítico.

Note, contudo, que, como $\theta(s) = \frac{e^s}{1 + e^s}$, então

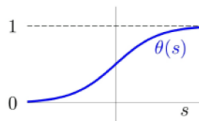
$$\theta(-s) = \frac{e^{-s}}{1 + e^{-s}} = \frac{1}{1 + e^s} = 1 - \frac{e^s}{1 + e^s} = 1 - \theta(s)$$

Regressão Logística - Medida de Erro

Assim, como $h(\vec{x}) = \theta(s)$, com $s = \vec{\omega} \cdot \vec{x}$

$$P(y|\vec{x}) = \begin{cases} \theta(s) & \text{para } y = +1 \\ \theta(-s) & \text{para } y = -1 \end{cases}$$

(observável no gráfico)



Note então que $P(y|\vec{x}) = \theta(y\vec{\omega} \cdot \vec{x})$:

- Como $y = \pm 1$, isso cobre os dois casos de $\theta(s)$ e $\theta(-s)$
- E, para o conjunto de dados $\mathcal{D} = (\vec{x}_1, y_1), \dots, (\vec{x}_N, y_N)$, a probabilidade de, sob a hipótese, observarmos cada saída, dado seu respectivo ponto, é:

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N P(y_n|\vec{x}_n) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

(assumindo-se independência entre os exemplos fornecidos)

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N P(y_n|\vec{x}_n) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

Note que o mesmo $\vec{\omega}$ que influencia em um ponto é o que influencia em todos

- O superajuste a um pode prejudicar o ajuste a outro

Queremos então encontrar $\vec{\omega}$ que maximiza essa probabilidade

- $\vec{\omega}$ que minimiza a medida do erro

Então, para maximizarmos

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

queremos o conjunto de $\vec{\omega}$ que minimize o erro

$$E(\vec{\omega}) = \frac{1}{N} \sum_{n=1}^N \ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n})$$

O chamado “cross-entropy error”

Regressão Logística - Maximização P

Queremos maximizar

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

Em relação a que?

Regressão Logística - Maximização P

Queremos maximizar

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

Em relação a que? A $\vec{\omega}$ (todo o resto é constante)

Regressão Logística - Maximização P

Queremos maximizar

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

Em relação a que? A $\vec{\omega}$ (todo o resto é constante)

Em vez disso, não poderíamos maximizar?

$$\ln(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) = \ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right)$$

Regressão Logística - Maximização P

Queremos maximizar

$$P(\mathcal{D}|h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

Em relação a que? A $\vec{\omega}$ (todo o resto é constante)

Em vez disso, não poderíamos maximizar?

$$\ln(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) = \ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right)$$

Certamente, pois $P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))$ não será negativa nem zero, e os mesmos parâmetros $\vec{\omega}$ que maximizam uma maximizam a outra

Ou seja:

$$\max(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) \iff \max \left(\ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right) \right)$$

E também nada nos impede de maximizar

$$\frac{1}{N} \ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right)$$

Note que estamos tentando chegar a uma expressão do erro...

Então, temos que:

$$\max(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) \Rightarrow \max \left(\frac{1}{N} \ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right) \right)$$

E podemos maximizar?

$$-\frac{1}{N} \ln \left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n) \right)$$

Então, temos que:

$$\max(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) \Rightarrow \max\left(\frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right)\right)$$

E podemos maximizar?

$$-\frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right)$$

Não. Mas se minimizarmos essa expressão, teremos maximizado sua correspondente positiva

Ou seja:

$$\max(P(\mathcal{D}|h(\vec{x}) = f(\vec{x}))) \iff \min\left(-\frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right)\right)$$

Como $\ln(a^b) = b \times \ln(a)$, então temos que

$$-\frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right) = \frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right)^{-1}$$

E como $(a \times b)^{-1} = a^{-1} \times b^{-1}$, então

$$\frac{1}{N} \ln\left(\prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)\right)^{-1} = \frac{1}{N} \ln\left(\prod_{n=1}^N \frac{1}{\theta(y_n \vec{\omega} \cdot \vec{x}_n)}\right)$$

E como $\ln(a \times b) = \ln(a) + \ln(b)$, então

$$\frac{1}{N} \ln \left(\prod_{n=1}^N \frac{1}{\theta(y_n \vec{\omega} \cdot \vec{x}_n)} \right) = \frac{1}{N} \sum_{n=1}^N \ln \left(\frac{1}{\theta(y_n \vec{\omega} \cdot \vec{x}_n)} \right)$$

Com $\theta(s) = \frac{e^s}{1 + e^s} = \frac{1}{1 + e^{-s}}$, temos que

$$\frac{1}{N} \sum_{n=1}^N \ln \left(\frac{1}{\theta(y_n \vec{\omega} \cdot \vec{x}_n)} \right) = \frac{1}{N} \sum_{n=1}^N \ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}) = E(\vec{\omega})$$

E, portanto, ao maximizarmos

$$P(\mathcal{D} | h(\vec{x}) = f(\vec{x})) = \prod_{n=1}^N \theta(y_n \vec{\omega} \cdot \vec{x}_n)$$

estaríamos minimizando o erro

$$E(\vec{\omega}) = \frac{1}{N} \sum_{n=1}^N \underbrace{\ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n})}_{\text{erro}(h(\vec{x}_n), y_n)}$$

Em nosso exemplo anterior...

$$E(\vec{\omega}) = \frac{1}{N} \sum_{n=1}^N \underbrace{\ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n})}_{\text{erro}(h(\vec{x}_n), y_n)}$$

Se o fator de risco $s_n = \vec{\omega} \cdot \vec{x}_n$ for grande, a chance de um ataque cardíaco é alta. Então, se $y = +1$ (ataque), $e^{-y_n \vec{\omega} \cdot \vec{x}_n}$ será minúsculo, e o erro se aproximará de zero ($\approx \ln(1)$)

Já se, com risco alto, o dado resultar em não-ataque ($y = -1$), então o exponencial será positivo, e o erro será enorme

Queremos minimizar esses erros

Regressão Linear

$$E(\vec{\omega}) = \frac{1}{N} \sum_{n=1}^N (\vec{\omega} \cdot \vec{x}_n - y_n)^2$$

Minimizado com o cálculo da pseudo inversa $X^\dagger \rightarrow \vec{\omega} = X^\dagger \vec{y}$

Regressão Logística

$$E(\vec{\omega}) = \frac{1}{N} \sum_{n=1}^N \ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n})$$

Como minimizar?

Regressão Logística - Minimizando o erro

Embora consigamos derivar uma forma fechada para a solução da regressão linear, o mesmo não é tão facilmente obtido com a regressão logística

Considere seu gradiente (em relação a $\vec{\omega}$)

$$\begin{aligned}\nabla E(\vec{\omega}) &= \nabla \left(\frac{1}{N} \sum_{n=1}^N \ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}) \right) \\ &= \frac{1}{N} \nabla \left(\sum_{n=1}^N \ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}) \right) \\ &= \frac{1}{N} \sum_{n=1}^N \nabla (\ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}))\end{aligned}$$

Regressão Logística - Minimizando o erro

$$\begin{aligned}\nabla E(\vec{\omega}) &= \frac{1}{N} \sum_{n=1}^N \nabla(\ln(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n})) \\&= \frac{1}{N} \sum_{n=1}^N \frac{1}{1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}} \nabla(1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}) \\&= \frac{1}{N} \sum_{n=1}^N \frac{1}{1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}} e^{-y_n \vec{\omega} \cdot \vec{x}_n} \nabla(-y_n \vec{\omega} \cdot \vec{x}_n) \\&= \frac{1}{N} \sum_{n=1}^N \frac{1}{1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}} e^{-y_n \vec{\omega} \cdot \vec{x}_n} (-y_n \vec{x}_n) \\&= -\frac{1}{N} \sum_{n=1}^N \frac{e^{-y_n \vec{\omega} \cdot \vec{x}_n}}{1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}} y_n \vec{x}_n\end{aligned}$$

Regressão Logística - Minimizando o erro

$$\begin{aligned}\nabla E(\vec{\omega}) &= -\frac{1}{N} \sum_{n=1}^N \frac{e^{-y_n \vec{\omega} \cdot \vec{x}_n}}{1 + e^{-y_n \vec{\omega} \cdot \vec{x}_n}} y_n \vec{x}_n \\ &= -\frac{1}{N} \sum_{n=1}^N \frac{1}{\frac{1}{e^{-y_n \vec{\omega} \cdot \vec{x}_n}} + 1} y_n \vec{x}_n\end{aligned}$$

e...

$$\nabla E(\vec{\omega}) = -\frac{1}{N} \sum_{n=1}^N \frac{y_n \vec{x}_n}{1 + e^{y_n \vec{\omega} \cdot \vec{x}_n}}$$

Regressão Logística - Minimizando o erro

$$\nabla E(\vec{\omega}) = -\frac{1}{N} \sum_{n=1}^N \frac{y_n \vec{x}_n}{1 + e^{y_n \vec{\omega} \cdot \vec{x}_n}}$$

- Regressão linear
 - forma fechada $\rightarrow$ solução obtida diretamente
- Regressão logística:
 - Essa solução não sai diretamente, por conta da exponencial
 - Solução iterativa $\rightarrow$ vamos melhorando-a passo a passo

Solução Iterativa - Gradient Descent

Vejam os a forma de $E(\vec{w})$:

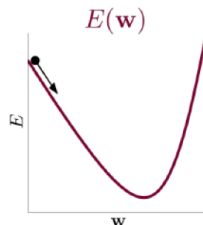
- Forma bastante amigável – há um único mínimo, em vez de vários

Podemos então usar Gradient Descent

- Método geral para otimização não-linear

O método consiste de:

- Iniciar em $\vec{w}_0$
- Dar um passo em direção à descida mais íngreme
 - Assumimos que não temos informação global, apenas local $\rightarrow$ não temos como ver o mínimo global da função



Solução Iterativa - Gradient Descent

Fazemos então essa aproximação, nos movendo no espaço de $\vec{\omega}$, em passos de tamanho fixo λ , em uma determinada direção $\hat{v}$ (vetor unitário). Então

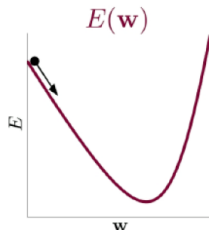
$$\vec{\omega}_1 = \vec{\omega}_0 + \lambda \hat{v}$$

Sob essas condições, tentamos descobrir a direção $\hat{v}$

Considere a variação do erro em $\vec{\omega}$:

$$\Delta E(\vec{\omega}) = E(\vec{\omega}_1) - E(\vec{\omega}_0)$$

Gostaríamos que $\Delta E(\vec{\omega})$ fosse o mais negativo possível (queremos minimizar o erro)



Solução Iterativa - Gradient Descent

Considere a expansão de Taylor de uma função $f(x)$ em torno do ponto a :

$$f(x) = f(a) + f'(a)(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n + \dots$$

Vamos expandir $E(\vec{\omega}_1)$ conforme a série de Taylor em torno do ponto $\vec{\omega}_0$:

$$E(\vec{\omega}_1) = E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\vec{\omega}_1 - \vec{\omega}_0) + \frac{\nabla^2 E(\vec{\omega}_0)}{2!} |\vec{\omega}_1 - \vec{\omega}_0|^2 + \dots$$

Solução Iterativa - Gradient Descent

$$E(\vec{\omega}_1) = E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\vec{\omega}_1 - \vec{\omega}_0) + \frac{\nabla^2 E(\vec{\omega}_0)}{2!} |\vec{\omega}_1 - \vec{\omega}_0|^2 + \dots$$

Como $\vec{\omega}_1 = \vec{\omega}_0 + \lambda \hat{v} \Rightarrow \vec{\omega}_1 - \vec{\omega}_0 = \lambda \hat{v}$, então

$$E(\vec{\omega}_1) = E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\lambda \hat{v}) + \frac{\nabla^2 E(\vec{\omega}_0)}{2!} |\lambda \hat{v}|^2 + \dots$$

Para simplificar, podemos considerar apenas os dois primeiros termos da expansão:

$$E(\vec{\omega}_1) = E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\lambda \hat{v}) + g(\lambda)$$

(Aproximação de primeira ordem (linear) da superfície)

Solução Iterativa - Gradient Descent

$$E(\vec{\omega}_1) = E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\lambda \hat{v}) + g(\lambda)$$

A expressão de $\Delta E(\vec{\omega})$ torna-se:

$$\begin{aligned}\Delta E(\vec{\omega}) &= E(\vec{\omega}_1) - E(\vec{\omega}_0) \\ &= E(\vec{\omega}_0) + \nabla E(\vec{\omega}_0) \cdot (\lambda \hat{v}) + g(\lambda) - E(\vec{\omega}_0) \\ &= \nabla E(\vec{\omega}_0) \cdot (\lambda \hat{v}) + g(\lambda) \\ &= \lambda(\nabla E(\vec{\omega}_0)) \cdot \hat{v} + g(\lambda)\end{aligned}$$

A suposição feita pelo Gradient Descent é que podemos ignorar $g(\lambda)$. Então:

$$\Delta E(\vec{\omega}) = \lambda(\nabla E(\vec{\omega}_0)) \cdot \hat{v}$$

Solução Iterativa - Gradient Descent

$$\Delta E(\vec{\omega}) = \lambda(\nabla E(\vec{\omega}_0)) \cdot \hat{v}$$

A questão então é como escolher a direção $\hat{v}$ de modo a fazer $\Delta E(\vec{\omega})$ o mais negativo possível?

A resposta se baseia na observação de que, para qualquer escolha de $\hat{v}$,

$$\lambda(\nabla E(\vec{\omega}_0)) \cdot \hat{v} \geq -\lambda|\nabla E(\vec{\omega}_0)|$$

Como $\hat{v}$ é unitário, então ele não contribui para a magnitude do resultado. Então, o menor valor que podemos conseguir será quando $\hat{v}$ estiver oposto a $\nabla E(\vec{\omega}_0)$, caso em que a magnitude será o tamanho de $\nabla E(\vec{\omega}_0)$, negativo.

Solução Iterativa - Gradient Descent

Assim, se escolhermos $\hat{v}$ em que a igualdade valha, achamos nosso $\hat{v}$, pois é o menor valor que conseguiremos para o erro.

$$\Delta E(\vec{\omega}) = \lambda(\nabla E(\vec{\omega}_0)) \cdot \hat{v} \geq -\lambda|\nabla E(\vec{\omega}_0)|$$

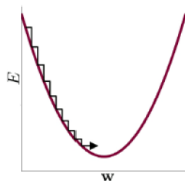
Isso é conseguido escolhendo-se um $\hat{v}$ (unitário) oposto a $\nabla E(\vec{\omega}_0)$:

$$\hat{v} = -\frac{\nabla E(\vec{\omega}_0)}{|\nabla E(\vec{\omega}_0)|}$$

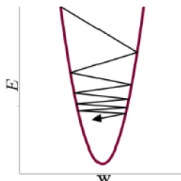
Estamos então descendo ao longo do gradiente do erro. Daí Gradient Descent.

Solução Iterativa - Gradient Descent

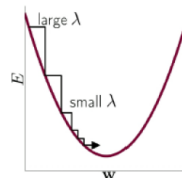
Como, no cálculo do gradiente, mantivemos apenas os termos lineares de Taylor, aproximamos cada passo por uma linha:



λ pequeno demais



λ grande demais



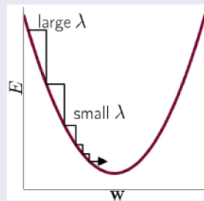
λ de tamanho variável

Embora com passos pequenos estejamos bem próximos à curva, o algoritmo levará muito tempo. Com passos largos, a aproximação poderá ser muito pobre.

Solução Iterativa - Gradient Descent

Podemos ter λ de tamanho variável

- Começamos com um λ grande
- À medida que nos aproximamos ao mínimo o reduzimos
- Assim λ reduziria com a redução da inclinação da curva



Implementação

Começemos com o passo fixo, lembrando que $\vec{\omega}_1 = \vec{\omega}_0 + \lambda \hat{v}$:

$$\begin{aligned}\Delta \vec{\omega} &= \lambda \hat{v} \\ &= -\lambda \frac{\nabla E(\vec{\omega}_0)}{|\nabla E(\vec{\omega}_0)|}\end{aligned}$$

Implementação

Se fizermos λ proporcional a $|\nabla E(\vec{\omega}_0)|$, ou seja, variando conforme o erro, teremos:

$$\lambda = \eta |\nabla E(\vec{\omega}_0)|$$

e

$$\Delta \vec{\omega} = -\lambda \frac{\nabla E(\vec{\omega}_0)}{|\nabla E(\vec{\omega}_0)|} = -\eta |\nabla E(\vec{\omega}_0)| \frac{\nabla E(\vec{\omega}_0)}{|\nabla E(\vec{\omega}_0)|} = -\eta \nabla E(\vec{\omega}_0)$$

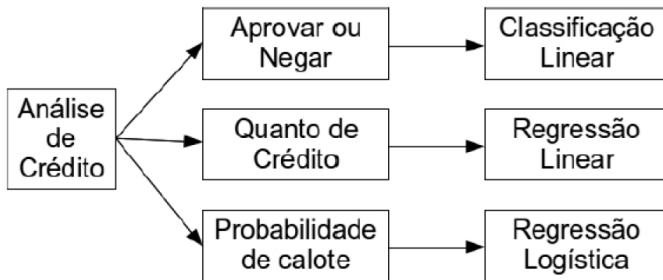
Com isso, o passo $\Delta \vec{\omega}$ não é mais fixo, mas sim η , chamada de **taxa de aprendizado**.

Algoritmo

- 1 Inicialize os pesos em t_0 com $\vec{\omega}_0$
- 2 Para $t = 0, 1, 2, \dots$ faça
- 3 Calcule o gradiente (slide 68)

$$\nabla E(\vec{\omega}) = -\frac{1}{N} \sum_{n=1}^N \frac{y_n \vec{x}_n}{1 + e^{y_n \vec{\omega}_t \cdot \vec{x}_n}}$$

- 4 Atualize os pesos: $\vec{\omega}_{t+1} = \vec{\omega}_t - \eta \nabla E(\vec{\omega}_t)$
- 5 Retorne os pesos finais $\vec{\omega}$



Seleção de Modelos e No Free Lunch Theorem

No contexto do aprendizado de máquina, estamos interessados em modelos que reduzam o erro de regressão ou classificação

- E como testar e comparar? Há vários modos...
 - Validação Cruzada
 - Modelos estatísticos de comparação, como correlação entre os resultados previstos e os reais, por exemplo
 - Etc...
- Serão vistos ao longo do curso

Qualquer que seja o modelo, alguns passos são necessários

- 1 Divida aleatoriamente o conjunto de dados em 2 subconjuntos: treinamento e teste
 - Ambos conjuntos precisam ser independentes
 - Nunca treine no de teste
- 2 Treine o modelo no conjunto de treinamento
- 3 Teste o modelo no conjunto de teste

Algumas vezes um terceiro conjunto – Validação – é incluído, servindo para ajustes finos do algoritmo durante o treinamento (de modo a evitar *overfitting*)

Uma vez que o teste definitivo se dará em dados reais, é importante que os conjuntos usados para treinamento e teste sejam representativos do todo.

Ainda assim, será que podemos dizer que um método é sempre melhor que outro?

No Free Lunch Theorem

Grupo de teoremas em que se prova, para várias situações, que, de fato, não se consegue aprender gratuitamente apenas olhando as instâncias de treinamento.

- Até porque o mundo real normalmente é bem maior

Ideia Básica

Se vemos a entrada $(0, 0)$ e $(1, 1)$, ambas classificadas como “falsas”, há duas hipóteses (dentre várias) que se adequam aos dados:

- Toda entrada será falsas
- Toda entrada, exceto essas duas, será verdadeira

Ou seja, dado um conjunto de treino, há sempre pelo menos duas generalizações igualmente plausíveis, mas totalmente opostas, que poderiam ser feitas.

- Todo algoritmo precisa de um viés para distinguir entre essas hipóteses
- Ex de viés: aprender apenas alguns tipos de funções (linear etc), introduzir restrições sobre o domínio do problema etc.

No Free Lunch Theorem

Mas fica pior...

Para todo algoritmo α , e para qualquer medida de desempenho, a média sobre todo problema de otimização f (seja função-custo ou função-objetivo) é independente de α

- O que um algoritmo ganha em desempenho em uma classe de problemas é necessariamente contra-balanceado por seu desempenho ruim nos demais problemas

Isso significa que o que se um algoritmo roda bem (em média) em uma classe de funções-objetivo, então ele será pior (em média) nas demais classes.

- Mais que isso, se um algoritmo tem melhor desempenho que seu equivalente aleatório em alguma classe de funções, então terá um desempenho pior nas demais

No Free Lunch Theorem

Ou seja...

Comparações de desempenho de um algoritmo particular, com valores particulares de parâmetros, em uns poucos problemas, são de uso limitado.

- Não existe modelo que funcione melhor para todo problema.
- O pressuposto de um ótimo modelo para um problema pode não valer para outro.
- É comum, em aprendizado, tentar vários modelos e ver qual o que funciona melhor para um problema particular.

No Free Lunch Theorem

e...

Da mesma forma que não há algoritmo universalmente eficaz, não há problema f , tanto de busca quanto otimização, que possa resultar em desempenho melhor que o aleatório, independentemente do algoritmo usado (ou seja, para todo e qualquer algoritmo usado).

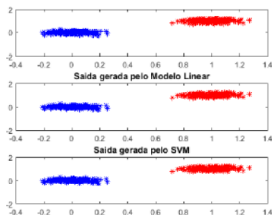
Resumindo

Não existem modelos genéricos de aprendizado de máquina que, em média, apresentem melhor desempenho que qualquer outro modelo para uma classe de problemas quaisquer.

Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 1

Considere duas distribuições uniformes com centro $[0,0]$ e $[1,1]$, ambas com desvio padrão $[0.1, 0.1]$.

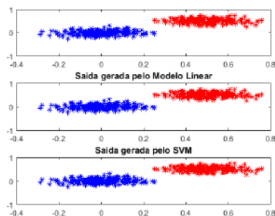


Comparação entre Modelo Linear versu SVM com Kernel Lineare

Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 2

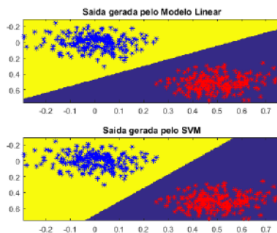
Considere duas distribuições uniformes com centro $[0,0]$ e $[0.5,0.5]$, ambas com desvio padrão $[0.1, 0.1]$.



Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 2

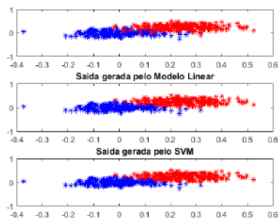
Fronteira de decisão



Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 3

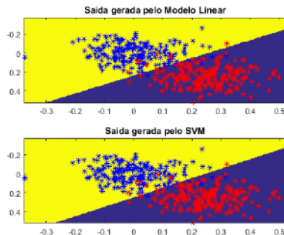
Considere duas distribuições uniformes com centro $[0.25, 0.25]$, ambas com desvio padrão $[0.1, 0.1]$.



Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 3

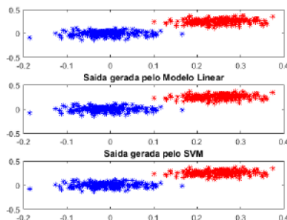
Fronteira de decisão



Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 4

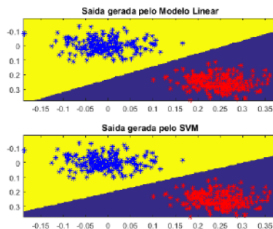
Considere duas distribuições uniformes com centro $[0,0]$ e $[0.25,0.25]$, ambas com desvio padrão $[0.05, 0.05]$.



Comparação entre Modelo Linear versu SVM com Kernel Lineare

Experimento # 4

Fronteira de decisão



- Alpaydm, E.: Introduction to Machine Learning. 2 ed. MIT Press. 2010.
- Harrington, P.: Machine Learning in Action. Manning. 2012.
- Ho, Y.C.; Pepyne, D.L.: Simple Explanation of the No Free Lunch Theorem of Optimization. Em Proceedings of the 40th IEEE Conference on Decision and Control. Orlando, FL, EUA. Dezembro, 2001.
- Machine Learning, Neural and Statistical Classification. D. Michie, D.J. Spiegelhalter, C.C. Taylor (eds.). 1994.
- Murphy, K. P.: Machine Learning: A Probabilistic Perspective. MIT Press. 2012.
- Wolpert, D. H.: The lack of a priori distinctions between learning algorithms. Em Neural Computation 8(7). Outubro, 1996.
- Wolpert, D.H.: What the No Free Lunch Theorems Really Mean: How to Improve Search Algorithms. SFI WORKING PAPER: 2012-10-017. Santa Fe Institute. Maio, 2012.

- Wolpert, D. H.; Macready, W. G.: No Free Lunch Theorems for Optimization. Em IEEE Transactions On Evolutionary Computation 1(1). Abril, 1997.
- <http://work.caltech.edu/library/>
- <http://www.cbc.umd.edu/~hcorrada/PracticalML/pdf/lectures/>
- http://www.aihorizon.com/essays/generalai/no_free_lunch_machine_learning.htm
- <http://www.statsblogs.com/2014/01/25/machine-learning-lesson-of-the-day-the-no-free-lunch-theorem/>

Obrigado!

Motivação e Intuição

- Extensão natural da regressão logística binária para C classes.
- Modelo prevê a probabilidade de cada classe:

$$P(Y = c \mid x) = \frac{\exp(w_c^\top x + b_c)}{\sum_{k=1}^C \exp(w_k^\top x + b_k)}$$

- As probabilidades são positivas e somam 1:

$$\sum_{c=1}^C P(Y = c \mid x) = 1$$

- O classificador final escolhe a classe com maior probabilidade:

$$\hat{y} = \arg \max_c P(Y = c \mid x)$$

Função de Perda e Regularização

- A função de custo é a **entropia cruzada**:

$$\mathcal{L}(W, b) = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log p_c(x_i)$$

onde $y_{i,c}$ é a codificação one-hot da classe.

- Com regularização L2 (Ridge):

$$\mathcal{L}_{reg} = \mathcal{L} + \frac{\lambda}{2} \sum_{c=1}^C \|w^{(c)}\|_2^2$$

- Gradiente (vetorial):

$$\nabla_W = \frac{1}{n} X^\top (P - Y) + \lambda W, \quad \nabla_b = \frac{1}{n} \mathbf{1}^\top (P - Y)$$

Pseudocódigo — Regressão Logística Multiclasse

Entrada: X ($n \times p$), y (rótulos $0..C-1$), taxa aprendizado , regularização
Inicializar W ($p \times C$) e b (C)

```
para it = 1 .. N_iter:
     $Z = X @ W + b$ 
     $Z -= \max(Z, \text{eixo}=1)$       # estabilidade numérica
     $P = \exp(Z) / \sum(\exp(Z), \text{eixo}=1)$ 
     $Y = \text{one\_hot}(y)$ 
     $\text{grad}W = X.T @ (P - Y) / n + W$ 
     $\text{grad}b = \text{mean}(P - Y, \text{eixo}=0)$ 
     $W -= \text{grad}W$ 
     $b -= \text{grad}b$ 
Retorna  $W, b$ 
```

Softmax garante probabilidades coerentes, e a regularização controla o overfitting.

Implementação do Zero em Python (NumPy)

```
import numpy as np

def softmax(z):
    z = z - np.max(z, axis=1, keepdims=True)
    expz = np.exp(z)
    return expz / np.sum(expz, axis=1, keepdims=True)

def one_hot(y, C):
    Y = np.zeros((len(y), C))
    Y[np.arange(len(y)), y] = 1
    return Y

def fit_softmax(X, y, lr=0.1, alpha=0.01, epochs=1000):
    n, p = X.shape
    C = np.unique(y).size
    W = np.zeros((p, C))
    b = np.zeros(C)
    Y = one_hot(y, C)
    for _ in range(epochs):
        P = softmax(X @ W + b)
        gradW = X.T @ (P - Y) / n + alpha * W
        gradb = np.mean(P - Y, axis=0)
        W -= lr * gradW
        b -= lr * gradb
```

Comparativo com Scikit-Learn

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

# Modelo Multinomial (softmax)
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(penalty='l2',
                      C=1.0,
                      solver='lbfgs',
                      multi_class='multinomial',
                      max_iter=1000)
)
model.fit(X_train, y_train)

# Comparação com o modelo do zero
W_manual, b_manual = fit_softmax(X_train, y_train)
y_pred_manual = np.argmax(softmax(X_test @ W_manual + b_manual), axis=1)
y_pred_sklearn = model.predict(X_test)
```

Comparação: Ambos otimizam a mesma função objetivo (softmax + entropia cruzada), mas o scikit-learn usa solvers otimizados como L-BFGS e regularização refinada.

- Regressão logística multiclasse é um modelo probabilístico linear:

$$P(Y = c \mid x) = \text{softmax}(W^T x + b)$$

- Usa-se **softmax** para garantir coerência entre classes.
- Implementações práticas (NumPy vs scikit-learn) diferem apenas na otimização.
- Regularização L2 é essencial para estabilidade e generalização.
- Softmax é base para redes neurais com múltiplas saídas (camada final).

- Métodos de otimização suportam ajustes de modelos lineares e kernelizados
- Lasso e Elastic Net requerem métodos iterativos
- Kernel trick permite trabalhar em espaços de alta dimensão sem computar $\phi(x)$
- Implementações manuais reproduzem resultados do scikit-learn